

Week6_예습과제

통계 수정 삭제

카사바 · 방금 전

❤️ 0

Euron_2026-1

▼ 목록 보기

5/5



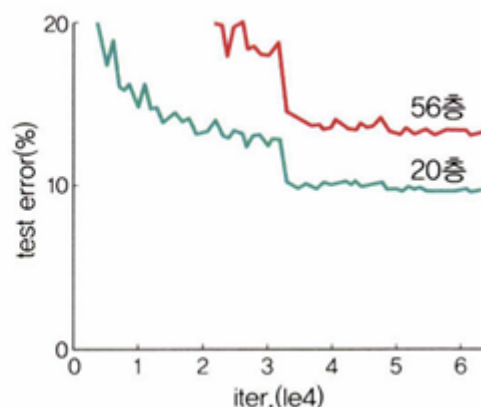
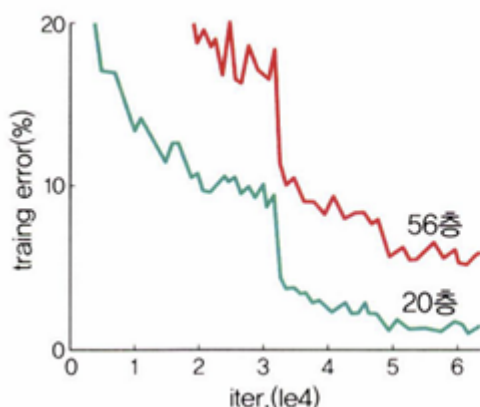
6장 합성곱 신경망II

6.1 이미지 분류를 위한 신경망

6.1.5 ResNet

ResNet: 마이크로소프트에서 개발한 알고리즘. 깊어진 신경망을 효과적으로 학습하기 위한 방법으로 **레지듀얼(residual)** 개념을 고안한 것이 핵심이다.

신경망 깊이가 깊어질수록 딥러닝 성능이 반드시 좋아지는 것은 아니다. 신경망은 깊이가 깊어질수록 성능이 좋아지다가 일정 단계에 다다르면 오히려 성능이 나빠진다.

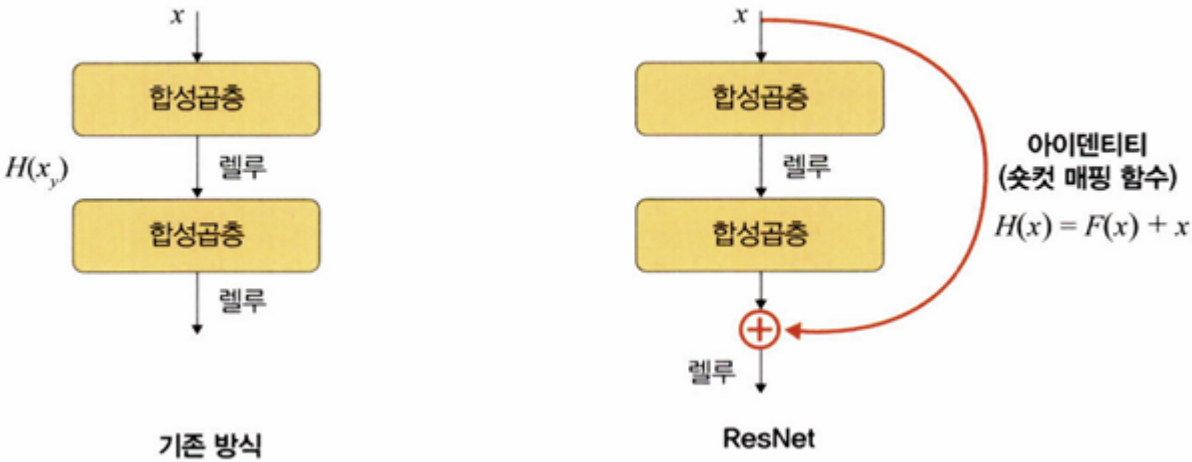


위 그림과 같이 네트워크 56층이 20층보다 나쁜 성능을 보인다.

ResNet은 이러한 문제를 해결하기 위해 residual block을 도입했다. 레지듀얼 블록은 기울기가 잘 전파될 수 있도록 일종의 숏컷(shortcut)을 만들어 준다.

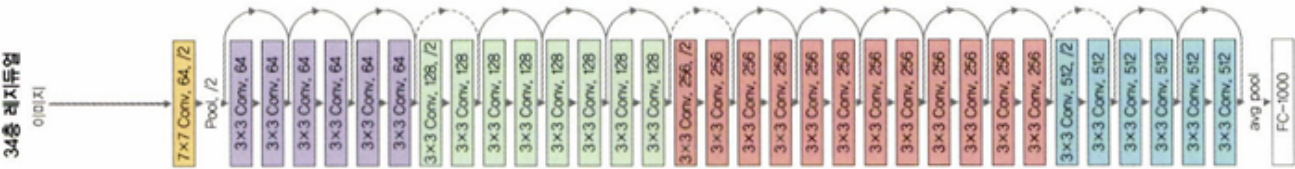
이러한 개념이 필요한 이유는 2014년 공개된 GoogLeNet은 층이 총 22개로 구성된 데 비해 ResNet은 층이 총 152개로 구성되어 기울기 소멸 문제가 발생할 수 있기 때문이다. 다음과 같이 숏컷을 두면 기울기 소멸 문제를 방지할 수 있다.

♥ 그림 6-25 ResNet 구조



블록(Block) : 계층의 묶음. 아래 그림에서 색상별로 블록을 구분했는데, 이렇게 묶인 계층들을 하나의 레지듀얼 블록이라고 한다. 그리고 레지듀얼 블록 여러 개를 쌓은 것을 ResNet이라고 한다.

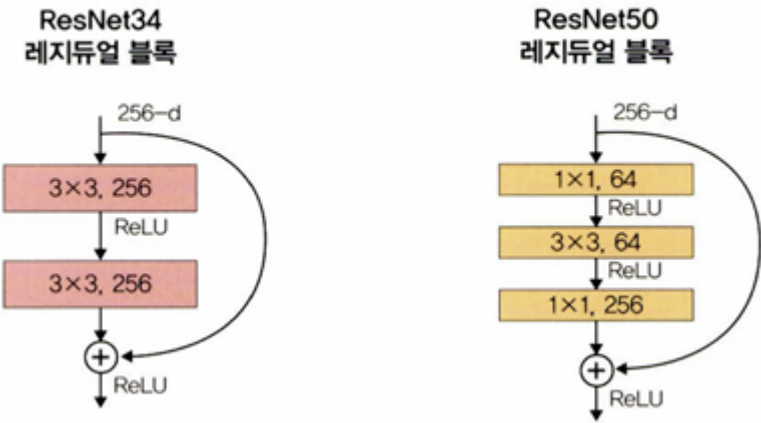
♥ 그림 6-26 ResNet 모델 전체 네트워크



하지만 이렇게 계층을 계속해서 쌓아 늘리면 파라미터 수가 문제가 된다. 계층이 깊어질수록 파라미터는 증가한다. 이러한 문제를 해결하기 위해 병목 블록(bottleneck block)이라는 것을 두었다.

ResNet34는 기본 블록(basic block)을 사용하며, ResNet50은 병목 블록을 사용한다. 기본 블록의 경우 파라미터 수가 39.3216M인 반면, 병목 블록의 경우 파라미터 수가 6.9632M이다. 깊이가 깊어졌음에도 파라미터 수가 감소한 것이다.

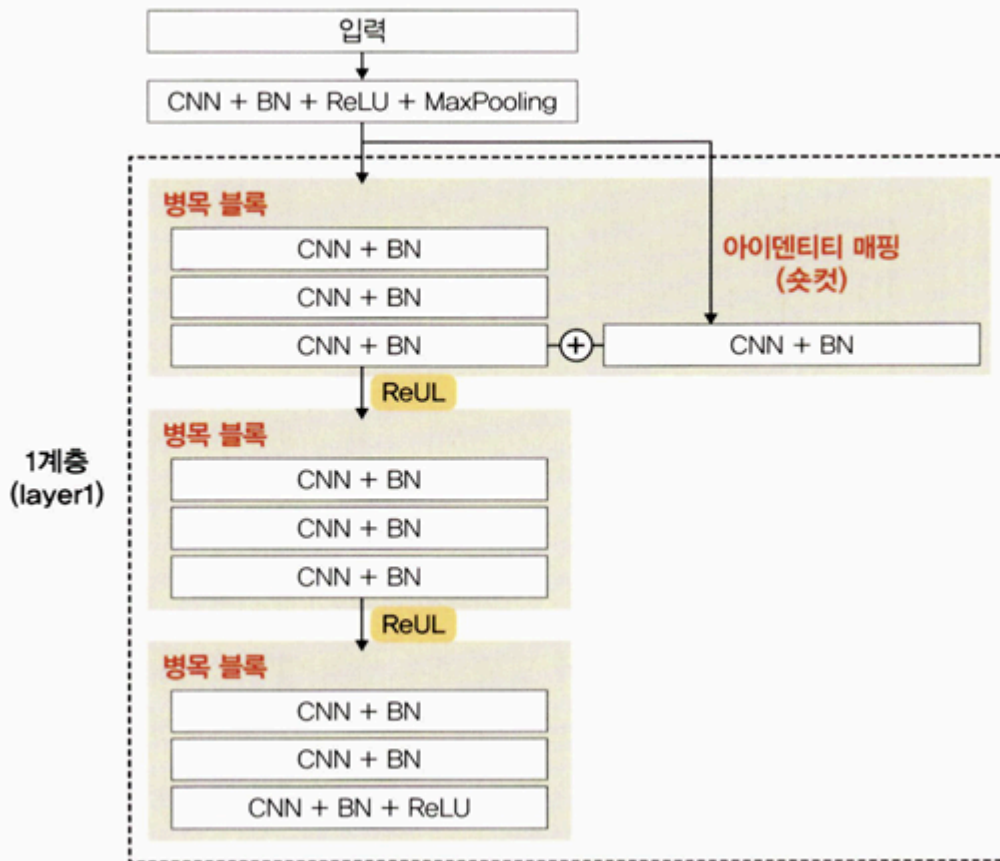
▼ 그림 6-27 기본 블록과 병목 블록



ResNet34와 다르게 ResNet50에서는 3x3 합성곱층 앞뒤로 1x1 합성곱층이 붙어 있는데, 1x1 합성곱층의 채널 수를 조절하면서 차원을 줄였다 늘리는 것이 가능하기 때문에 파라미터 수를 줄일 수 있었던 것이다.

아이덴티티 매핑(혹은 숏컷, 스킵 연결) : 위 그림 아래쪽에 있는 + 기호가 나타내는 부분.
아이덴티티 매핑은 입력 x 가 어떤 함수를 통과하더라도 다시 x 라는 형태로 출력되도록 한다.

▼ 그림 6-28 아이덴티티 매핑(숏컷)



```
def forward(self, x):
    i = x
    x = self.conv1(x)
    x = self.bn1(x)
    x = self.relu(x)
    x = self.conv2(x)
    x = self.bn2(x)

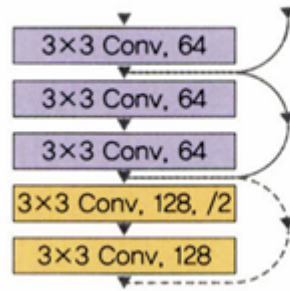
    if self.downsample is not None:
        i = self.downsample(i) # 다운샘플 적용

    x += i # 아이덴티티 매핑 적용
    x = self.relu(x)
    return x
```

예를 들어 x 가 (28, 28, 64)라고 해보자. x 가 합성곱층을 통과하면서 같은 형태를 더하기 때문에 최종 형태는 (28, 28, 64) 그대로가 될 것이다.

다운샘플(downsample) : 특성 맵 크기를 줄이기 위한 것으로, 풀링과 같은 역할을 한다.

▼ 그림 6-29 ResNet 네트워크의 일부



보라색 영역의 첫 번째 블록에서 특성 맵의 형상이 (28, 28, 64)였다면 세 번째 블록의 마지막 합성곱층을 통과하고 아이덴티티 매핑까지 완료된 특성 맵의 형상도 (28, 28, 64)이다.

노란색 영역의 시작 지점에서는 채널 수가 128로 늘어났고, 첫 번째 블록에서 합성곱층의 스트라이드가 2로 늘어나 (14, 14, 128)로 바뀐다는 것을 알 수 있다.

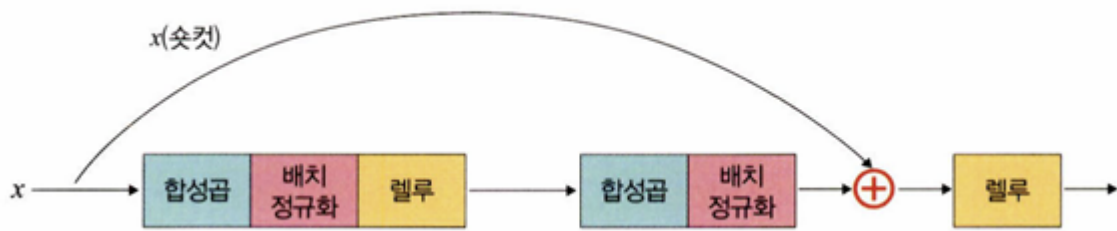
즉, 보라색과 노란색의 형태가 다른데 이들 간의 형태를 맞추지 않으면 아이덴티티 매핑을 할 수 없게 된다. 그래서 아이덴티티에 대해 다운샘플이 필요하다.

입력과 출력의 형태를 같도록 맞추어 주기 위해서는 스트라이드 2를 가진 1x1 합성곱 계층을 하나 연결해 주면 된다. 이와 같이 입력과 출력의 차원이 같은 것을 아이덴티티 블록이라고 하며, 입력 및 출력 차원이 동일하지 않고 입력의 차원을 출력에 맞추어 변경해야 하는 것을 프로젝션 숏컷 혹은 합성곱 블록이라고 한다.

▼ 그림 6-30 합성곱 블록

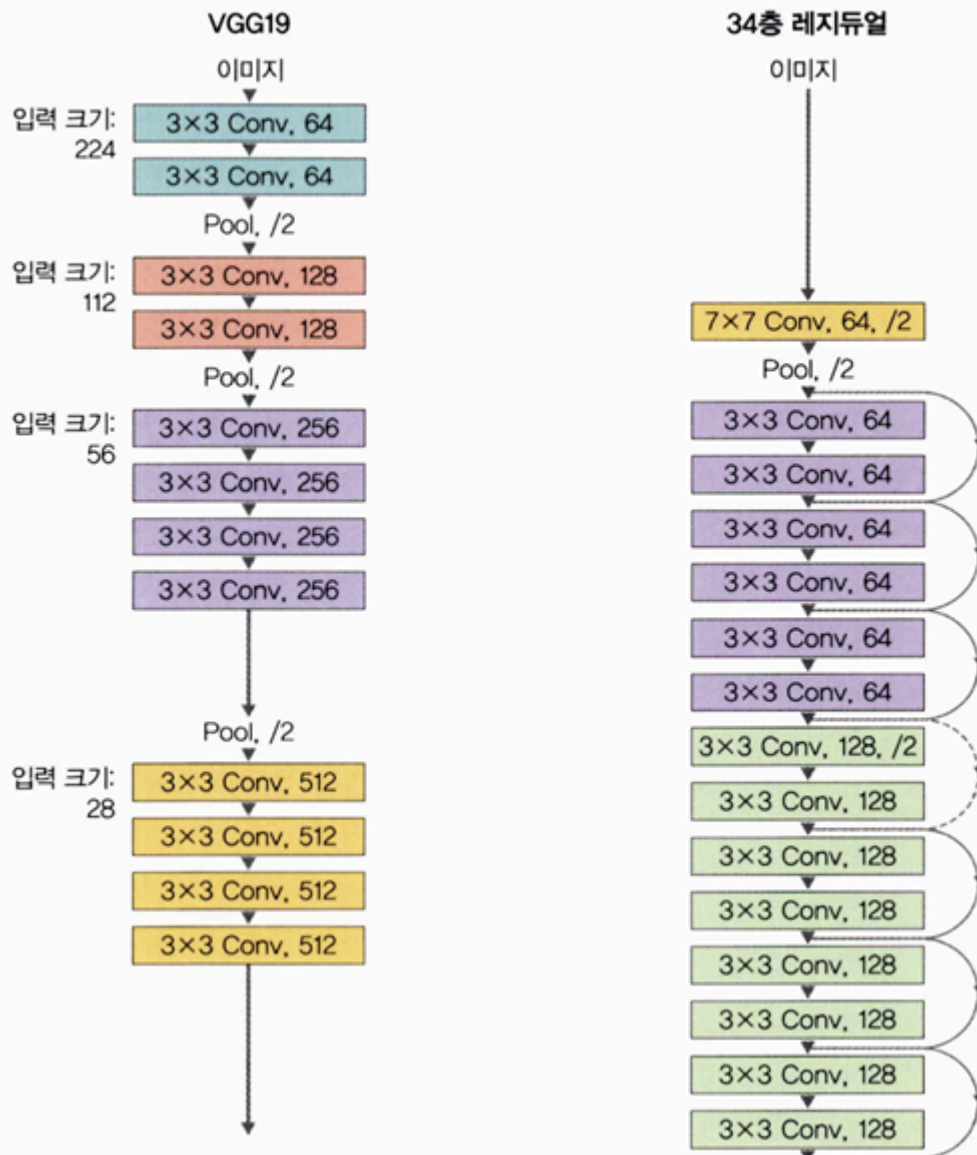


▼ 그림 6-31 아이덴티티 블록



ResNet은 기본적으로 VGG19 구조를 뼈대로 하며, 거기에 합성곱층들을 추가해서 깊게 만든 후 숏컷들을 추가한다.

▼ 그림 6-32 VGG19와 ResNet 비교



필요한 라이브러리 호출

```
import torch
import torch.nn as nn
import torch.nn.functional as F
import torch.optim as optim
import torch.utils.data as data
import torchvision
import torchvision.transforms as transforms
import torchvision.datasets as datasets
import torchvision.models as models

import matplotlib.pyplot as plt
import numpy as np

import copy
from collections import namedtuple # ①
import os
import random
import time

import cv2
from torch.utils.data import DataLoader, Dataset
from PIL import Image

device = torch.device('cuda' if torch.cuda.is_available() else 'cpu')
```

① 네임드튜플은 파이썬의 자료형 중 하나이다. 튜플의 성질을 갖고 있는 자료형이지만 인덱스뿐만 아니라 키 값으로 데이터에 접근할 수 있다. 따라서 다음과 같이 사용 가능하다.

```
from collections import namedtuple
Student = namedtuple('Student', ['name', 'age', 'DOB']) # 네임드튜플 정의
S = Student('홍길동', '19', '187') # 네임드튜플에 값 추가

print("The Student age using index is : ", end="")
print(S[1]) # 인덱스를 이용한 데이터 접근

print("The Student name using keyname is : ", end="")
print(S.name) # 키 값을 이용한 데이터 접근
```

네임드튜플에 대한 출력 결과는 다음과 같다.

```
The Student age using index is : 19
The Student name using keyname is : 홍길동
```

이미지 데이터 전처리

```
class ImageTransform():
    def __init__(self, resize, mean, std):
        self.data_transform = {
            'train': transforms.Compose([
                transforms.RandomResizedCrop(resize, scale=(0.5, 1.0)),
                transforms.RandomHorizontalFlip(),
                transforms.ToTensor(),
                transforms.Normalize(mean, std)
            ]), # 훈련 이미지 데이터에 대한 전처리
            'val': transforms.Compose([
                transforms.Resize(256),
                transforms.CenterCrop(resize),
                transforms.ToTensor(),
                transforms.Normalize(mean, std)
            ]) # 검증과 테스트 이미지 데이터에 대한 전처리
        }

    def __call__(self, img, phase):
        return self.data_transform[phase](img)
```

변수에 대한 값 정의

```
size = 224
mean = (0.485, 0.456, 0.406)
std = (0.229, 0.224, 0.225)
batch_size = 32
```

torchvision.datasets.ImageFolder를 이용해 훈련과 테스트 데이터셋을 불러온다.

훈련과 테스트 데이터셋 불러오기

```
cat_directory = r"/content/drive/MyDrive/data/dogs-vs-cats/Cat/"
dog_directory = r"/content/drive/MyDrive/data/dogs-vs-cats/Dog/"

cat_images_filepaths = sorted([os.path.join(cat_directory, f) for f in os.listdir(cat_directory)])
dog_images_filepaths = sorted([os.path.join(dog_directory, f) for f in os.listdir(dog_directory)])
images_filepaths = [*cat_images_filepaths, *dog_images_filepaths]
correct_images_filepaths = [i for i in images_filepaths if cv2.imread(i) is not None]
```

데이터셋을 훈련, 검증, 테스트 용도로 분리

```
random.seed(42)
random.shuffle(correct_images_filepaths)
train_images_filepaths = correct_images_filepaths[:400]
val_images_filepaths = correct_images_filepaths[400:-10]
test_images_filepaths = correct_images_filepaths[-10:]
```



```
print(len(train_images_filepaths), len(val_images_filepaths),  
      len(test_images_filepaths))
```

```
400 92 10
```

이미지에 대한 레이블 구분

```
class DogvsCatDataset(Dataset):  
    def __init__(self, file_list, transform=None, phase='train'):  
        self.file_list = file_list  
        self.transform = transform  
        self.phase = phase  
  
    def __len__(self):  
        return len(self.file_list)  
  
    def __getitem__(self, idx):  
        img_path = self.file_list[idx]  
        img = Image.open(img_path)  
        img_transformed = self.transform(img, self.phase)  
  
        label = img_path.split('/')[-1].split('.')[0]  
        if label == 'dog':  
            label = 1  
        elif label == 'cat':  
            label = 0  
        return img_transformed, label
```

이미지 데이터셋 정의

```
train_dataset = DogvsCatDataset(train_images_filepaths, transform = ImageTransform(size, mean,  
val_dataset = DogvsCatDataset(val_images_filepaths, transform = ImageTransform(size, mean, std  
  
index = 0  
print(train_dataset.__getitem__(index)[0].size())  
print(train_dataset.__getitem__(index)[1])
```

다음은 훈련 데이터셋 index 0의 이미지 크기와 레이블에 대한 출력 결과이다.

```
torch.Size([3, 224, 224])  
0
```

이미지는 컬러이고 224x224 크기이며, 레이블이 0이므로 고양이를 의미한다.

데이터셋의 데이터를 메모리로 불러오기

```
train_iterator = DataLoader(train_dataset, batch_size=batch_size, shuffle=True)
valid_iterator = DataLoader(val_dataset, batch_size=batch_size, shuffle=False)
dataloader_dict = {'train': train_iterator, 'val': valid_iterator}
```

```
batch_iterator = iter(train_iterator)
inputs, label = next(batch_iterator)
print(inputs.size())
print(label)
```

```
torch.Size([32, 3, 224, 224]) tensor([1, 1, 0, 0, 1, 1, 1, 0, 1, 1, 1, 0, 0, 0, 1,
0, 1, 0, 1, 1, 0, 1, 1, 0, 0, 1, 1, 1, 1, 0, 1, 1])
```

이제 ResNet의 전체 네트워크 구성을 위해 그것을 구성하는 기본 블록과 병목 블록에 대한 코드를 작성한다. 기본 블록은 ResNet18, ResNet34에서 사용되며 합성곱(3x3) 두 개로 구성된다.

기본 블록 정의

```
class BasicBlock(nn.Module):
    expansion = 1

    def __init__(self, in_channels, out_channels, stride=1, downsample=False):
        super().__init__()
        self.conv1 = nn.Conv2d(in_channels, out_channels, kernel_size=3,
                                stride=stride, padding=1, bias=False) # 3x3 합성곱층
        self.bn1 = nn.BatchNorm2d(out_channels)
        self.conv2 = nn.Conv2d(out_channels, out_channels, kernel_size=3,
                                stride=1, padding=1, bias=False) # 3x3 합성곱층
        self.bn2 = nn.BatchNorm2d(out_channels)
        self.relu = nn.ReLU(inplace=True)

        if downsample:
            conv = nn.Conv2d(in_channels, out_channels, kernel_size=1,
                              stride=stride, bias=False)
            bn = nn.BatchNorm2d(out_channels)
            downsample = nn.Sequential(conv, bn)
        else:
            downsample = None
        self.downsample = downsample

    def forward(self, x):
        i = x
        x = self.conv1(x)
        x = self.bn1(x)
        x = self.relu(x)
        x = self.conv2(x)
        x = self.bn2(x)

        if self.downsample is not None:
```

```

        i = self.downsample(i)

    x += i
    x = self.relu(x)

    return x

```

병목 블록은 ResNet50, ResNet101, ResNet152에서 사용되며 1x1 합성곱층, 3x3 합성곱층, 1x1 합성곱층으로 구성된다.

병목 블록 정의

```

class Bottleneck(nn.Module):
    expansion = 4 # ResNet에서 병목 블록을 정의하기 위한 하이퍼파라미터

    def __init__(self, in_channels, out_channels, stride=1, downsample=False):
        super().__init__()
        self.conv1 = nn.Conv2d(in_channels, out_channels, kernel_size=1,
                                stride=1, bias=False) # 1x1 합성곱층
        self.bn1 = nn.BatchNorm2d(out_channels)
        self.conv2 = nn.Conv2d(out_channels, out_channels, kernel_size=3,
                                stride=stride, padding=1, bias=False) # 3x3 합성곱층
        self.bn2 = nn.BatchNorm2d(out_channels)
        self.conv3 = nn.Conv2d(out_channels, self.expansion*out_channels,
                                kernel_size=1, stride=1, bias=False) # 1x1 합성곱층 또한 다음 계층의 입
        self.bn3 = nn.BatchNorm2d(self.expansion*out_channels)
        self.relu = nn.ReLU(inplace=True)

        if downsample:
            conv = nn.Conv2d(in_channels, self.expansion*out_channels, kernel_size=1,
                              stride=stride, bias=False)
            bn = nn.BatchNorm2d(self.expansion*out_channels)
            downsample = nn.Sequential(conv, bn)
        else:
            downsample = None
        self.downsample = downsample

    def forward(self, x):
        i = x
        x = self.conv1(x)
        x = self.bn1(x)
        x = self.relu(x)
        x = self.conv2(x)
        x = self.bn2(x)
        x = self.relu(x)
        x = self.conv3(x)
        x = self.bn3(x)

        if self.downsample is not None:
            i = self.downsample(i)

        x += i
        x = self.relu(x)
        return x

```

기본 블록은 3x3 합성곱층 두 개를 갖는 반면, 병목 블록은 1x1 합성곱층, 3x3 합성곱층, 1x1 합성곱층의 구조를 갖는다. 기본 블록을 병목 블록으로 변경하는 이유는 계층을 더 깊게 쌓으면서 계산 비용을 줄일 수 있기 때문이다. 그리고 계층이 많아진다는 것은 곧 활성화 함수가 기존보다 더 많이 포함된다는 것이고, 이것은 더 많은 비선형성을 처리할 수 있음을 의미한다. 즉, 다양한 입력 데이터에 대한 처리가 가능하다.

ResNet 모델 네트워크

```
class ResNet(nn.Module):
    def __init__(self, config, output_dim, zero_init_residual=False):
        super().__init__()

        block, n_blocks, channels = config # ResNet을 호출할 때 넘겨준 config 값들을 block, n_blocks,
        self.in_channels = channels[0]
        assert len(n_blocks) == len(channels) == 4 # 블록 크기=채널 크기=4

        self.conv1 = nn.Conv2d(3, self.in_channels, kernel_size=7, stride=2, padding=3, bias=False)
        self.bn1 = nn.BatchNorm2d(self.in_channels)
        self.relu = nn.ReLU(inplace=True)
        self.maxpool = nn.MaxPool2d(kernel_size=3, stride=2, padding=1)

        self.layer1 = self.get_resnet_layer(block, n_blocks[0], channels[0])
        self.layer2 = self.get_resnet_layer(block, n_blocks[1], channels[1], stride=2)
        self.layer3 = self.get_resnet_layer(block, n_blocks[2], channels[2], stride=2)
        self.layer4 = self.get_resnet_layer(block, n_blocks[3], channels[3], stride=2)

        self.avgpool = nn.AdaptiveAvgPool2d((1,1))
        self.fc = nn.Linear(self.in_channels, output_dim)

        if zero_init_residual: # ㉠
            for m in self.modules():
                if isinstance(m, Bottleneck):
                    nn.init.constant_(m.bn3.weight, 0)
                elif isinstance(m, BasicBlock):
                    nn.init.constant_(m.bn2.weight, 0)

    def get_resnet_layer(self, block, n_blocks, channels, stride=1): # 블록을 추가하기 위한 함수
        layers = []
        if self.in_channels != block.expansion * channels: # in_channels와 block.expansion*channels
            downsample = True
        else:
            downsample = False

        layers.append(block(self.in_channels, channels, stride, downsample)) # 계층을 추가할 때 in_cha
        for i in range(1, n_blocks): # n_blocks만큼 계층 추가
            layers.append(block(block.expansion*channels, channels))

        self.in_channels = block.expansion*channels
        return nn.Sequential(*layers)

    def forward(self, x):
        x = self.conv1(x) # 224x224
        x = self.bn1(x)
        x = self.relu(x)
```

```

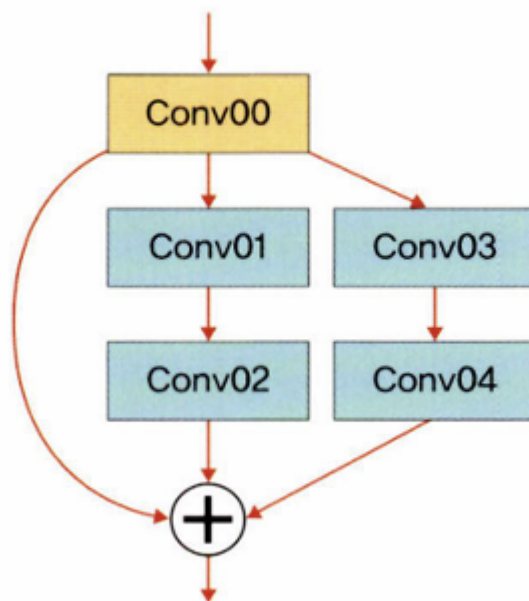
x = self.maxpool(x) # 112x112
x = self.layer1(x) # 56x56
x = self.layer2(x) # 28x28
x = self.layer3(x) # 14x14
x = self.layer4(x) # 7x7
x = self.avgpool(x) # 1x1
h = x.view(x.shape[0], -1)
x = self.fc(h)
return x, h

```

① 각 레지듀얼 분기에 있는 마지막 BN을 0으로 초기화해서 다음 레지듀얼 분기를 0에서 시작할 수 있도록 한다. 이 부분은 모델을 생성하고 학습시키는 것과는 상관없지만, BN을 0으로 초기화할 경우 모델 성능이 0.2~0.3%정도 향상된다고 한다. 따라서 ResNet에서는 많이 사용되고 있다.

레지듀얼 분기란 프로그램에서의 조건에 따라 A, B, C 등으로 분기하는 것과 같다.

▼ 그림 6-33 레지듀얼 분기



ResNetConfig 정의

```

ResNetConfig = namedtuple('ResNetConfig', ['block', 'n_blocks', 'channels'])

```

기본 블록을 사용하는 ResNet18과 ResNet34의 Config를 정의한다. 블록은 기본 블록을 사용하도록 하며, 블록(n_blocks)과 채널(channels)의 크기를 각각 지정한다.

기본 블록을 사용하여 ResNetConfig 정의

```
resnet18_config = ResNetConfig(block=BasicBlock,
                                n_blocks=[2,2,2,2],
                                channels=[64,128,256,512])

resnet34_config = ResNetConfig(block=BasicBlock,
                                n_blocks=[3,4,6,3],
                                channels=[64,128,256,512])
```

병목 블록을 사용하는 ResNet50, ResNet101, ResNet152의 Config를 정의한다. 블록은 병목 블록을 사용하며, 블록(n_blocks)과 채널(channels)의 크기를 각각 지정한다.

병목 블록을 사용하여 ResNetConfig 정의

```
resnet50_config = ResNetConfig(block=Bottleneck,
                                n_blocks=[3,4,6,3],
                                channels=[64,128,256,512])

resnet101_config = ResNetConfig(block=Bottleneck,
                                n_blocks=[3,4,23,3],
                                channels=[64,128,256,512])

resnet152_config = ResNetConfig(block=Bottleneck,
                                n_blocks=[3,8,36,3],
                                channels=[64,128,256,512])
```

사전 훈련된 ResNet 모델 사용

```
pretrained_model = models.resnet50(pretrained=True) # 사전 훈련된 ResNet 모델을 위해서는 pret
```

사전 훈련된 ResNet 네트워크 확인

```
print(pretrained_model)
```

```
ResNet( (conv1): Conv2d(3, 64, kernel_size=(7, 7), stride=(2, 2), padding=(3, 3),
bias=False) (bn1): BatchNorm2d(64, eps=1e-05, momentum=0.1, affine=True,
track_running_stats=True) (relu): ReLU(inplace=True) (maxpool):
MaxPool2d(kernel_size=3, stride=2, padding=1, dilation=1, ceil_mode=False) (layer1):
Sequential( (0): Bottleneck( (conv1): Conv2d(64, 64, kernel_size=(1, 1), stride=(1,
1), bias=False) (bn1): BatchNorm2d(64, eps=1e-05, momentum=0.1, affine=True,
track_running_stats=True) (conv2): Conv2d(64, 64, kernel_size=(3, 3), stride=(1, 1),
padding=(1, 1), bias=False) (bn2): BatchNorm2d(64, eps=1e-05, momentum=0.1,
affine=True, track_running_stats=True) (conv3): Conv2d(64, 256, kernel_size=(1, 1),
```

```

stride=(1, 1), bias=False) (bn3): BatchNorm2d(256, eps=1e-05, momentum=0.1,
affine=True, track_running_stats=True) (relu): ReLU(inplace=True) (downsample):
Sequential( (0): Conv2d(64, 256, kernel_size=(1, 1), stride=(1, 1), bias=False) (1):
BatchNorm2d(256, eps=1e-05, momentum=0.1, affine=True, track_running_stats=True) ) )
(1): Bottleneck( (conv1): Conv2d(256, 64, kernel_size=(1, 1), stride=(1, 1),
bias=False) (bn1): BatchNorm2d(64, eps=1e-05, momentum=0.1, affine=True,
track_running_stats=True) (conv2): Conv2d(64, 64, kernel_size=(3, 3), stride=(1, 1),
padding=(1, 1), bias=False) (bn2): BatchNorm2d(64, eps=1e-05, momentum=0.1,
affine=True, track_running_stats=True) (conv3): Conv2d(64, 256, kernel_size=(1, 1),
stride=(1, 1), bias=False) (bn3): BatchNorm2d(256, eps=1e-05, momentum=0.1,
affine=True, track_running_stats=True) (relu): ReLU(inplace=True) ) (2): Bottleneck(
(conv1): Conv2d(256, 64, kernel_size=(1, 1), stride=(1, 1), bias=False) (bn1):
BatchNorm2d(64, eps=1e-05, momentum=0.1, affine=True, track_running_stats=True)
(conv2): Conv2d(64, 64, kernel_size=(3, 3), stride=(1, 1), padding=(1, 1),
bias=False) (bn2): BatchNorm2d(64, eps=1e-05, momentum=0.1, affine=True,
track_running_stats=True) (conv3): Conv2d(64, 256, kernel_size=(1, 1), stride=(1,
1), bias=False) (bn3): BatchNorm2d(256, eps=1e-05, momentum=0.1, affine=True,
track_running_stats=True) (relu): ReLU(inplace=True) ) ) (layer2): Sequential( (0):
Bottleneck( (conv1): Conv2d(256, 128, kernel_size=(1, 1), stride=(1, 1), bias=False)
(bn1): BatchNorm2d(128, eps=1e-05, momentum=0.1, affine=True,
track_running_stats=True) (conv2): Conv2d(128, 128, kernel_size=(3, 3), stride=(2,
2), padding=(1, 1), bias=False) (bn2): BatchNorm2d(128, eps=1e-05, momentum=0.1,
affine=True, track_running_stats=True) (conv3): Conv2d(128, 512, kernel_size=(1, 1),
stride=(1, 1), bias=False) (bn3): BatchNorm2d(512, eps=1e-05, momentum=0.1,
affine=True, track_running_stats=True) (relu): ReLU(inplace=True) (downsample):
Sequential( (0): Conv2d(256, 512, kernel_size=(1, 1), stride=(2, 2), bias=False)
(1): BatchNorm2d(512, eps=1e-05, momentum=0.1, affine=True,
track_running_stats=True) ) ) (1): Bottleneck( (conv1): Conv2d(512, 128,
kernel_size=(1, 1), stride=(1, 1), bias=False) (bn1): BatchNorm2d(128, eps=1e-05,
momentum=0.1, affine=True, track_running_stats=True) (conv2): Conv2d(128, 128,
kernel_size=(3, 3), stride=(1, 1), padding=(1, 1), bias=False) (bn2):
BatchNorm2d(128, eps=1e-05, momentum=0.1, affine=True, track_running_stats=True)
(conv3): Conv2d(128, 512, kernel_size=(1, 1), stride=(1, 1), bias=False) (bn3):
BatchNorm2d(512, eps=1e-05, momentum=0.1, affine=True, track_running_stats=True)
(relu): ReLU(inplace=True) ) (2): Bottleneck( (conv1): Conv2d(512, 128, kernel_size=
(1, 1), stride=(1, 1), bias=False) (bn1): BatchNorm2d(128, eps=1e-05, momentum=0.1,
affine=True, track_running_stats=True) (conv2): Conv2d(128, 128, kernel_size=(3, 3),
stride=(1, 1), padding=(1, 1), bias=False) (bn2): BatchNorm2d(128, eps=1e-05,

```

```

momentum=0.1, affine=True, track_running_stats=True) (conv3): Conv2d(128, 512,
kernel_size=(1, 1), stride=(1, 1), bias=False) (bn3): BatchNorm2d(512, eps=1e-05,
momentum=0.1, affine=True, track_running_stats=True) (relu): ReLU(inplace=True) )
(3): Bottleneck( (conv1): Conv2d(512, 128, kernel_size=(1, 1), stride=(1, 1),
bias=False) (bn1): BatchNorm2d(128, eps=1e-05, momentum=0.1, affine=True,
track_running_stats=True) (conv2): Conv2d(128, 128, kernel_size=(3, 3), stride=(1,
1), padding=(1, 1), bias=False) (bn2): BatchNorm2d(128, eps=1e-05, momentum=0.1,
affine=True, track_running_stats=True) (conv3): Conv2d(128, 512, kernel_size=(1, 1),
stride=(1, 1), bias=False) (bn3): BatchNorm2d(512, eps=1e-05, momentum=0.1,
affine=True, track_running_stats=True) (relu): ReLU(inplace=True) ) ) (layer3):
Sequential( (0): Bottleneck( (conv1): Conv2d(512, 256, kernel_size=(1, 1), stride=
(1, 1), bias=False) (bn1): BatchNorm2d(256, eps=1e-05, momentum=0.1, affine=True,
track_running_stats=True) (conv2): Conv2d(256, 256, kernel_size=(3, 3), stride=(2,
2), padding=(1, 1), bias=False) (bn2): BatchNorm2d(256, eps=1e-05, momentum=0.1,
affine=True, track_running_stats=True) (conv3): Conv2d(256, 1024, kernel_size=(1,
1), stride=(1, 1), bias=False) (bn3): BatchNorm2d(1024, eps=1e-05, momentum=0.1,
affine=True, track_running_stats=True) (relu): ReLU(inplace=True) (downsample):
Sequential( (0): Conv2d(512, 1024, kernel_size=(1, 1), stride=(2, 2), bias=False)
(1): BatchNorm2d(1024, eps=1e-05, momentum=0.1, affine=True,
track_running_stats=True) ) ) (1): Bottleneck( (conv1): Conv2d(1024, 256,
kernel_size=(1, 1), stride=(1, 1), bias=False) (bn1): BatchNorm2d(256, eps=1e-05,
momentum=0.1, affine=True, track_running_stats=True) (conv2): Conv2d(256, 256,
kernel_size=(3, 3), stride=(1, 1), padding=(1, 1), bias=False) (bn2):
BatchNorm2d(256, eps=1e-05, momentum=0.1, affine=True, track_running_stats=True)
(conv3): Conv2d(256, 1024, kernel_size=(1, 1), stride=(1, 1), bias=False) (bn3):
BatchNorm2d(1024, eps=1e-05, momentum=0.1, affine=True, track_running_stats=True)
(relu): ReLU(inplace=True) ) (2): Bottleneck( (conv1): Conv2d(1024, 256,
kernel_size=(1, 1), stride=(1, 1), bias=False) (bn1): BatchNorm2d(256, eps=1e-05,
momentum=0.1, affine=True, track_running_stats=True) (conv2): Conv2d(256, 256,
kernel_size=(3, 3), stride=(1, 1), padding=(1, 1), bias=False) (bn2):
BatchNorm2d(256, eps=1e-05, momentum=0.1, affine=True, track_running_stats=True)
(conv3): Conv2d(256, 1024, kernel_size=(1, 1), stride=(1, 1), bias=False) (bn3):
BatchNorm2d(1024, eps=1e-05, momentum=0.1, affine=True, track_running_stats=True)
(relu): ReLU(inplace=True) ) (3): Bottleneck( (conv1): Conv2d(1024, 256,
kernel_size=(1, 1), stride=(1, 1), bias=False) (bn1): BatchNorm2d(256, eps=1e-05,
momentum=0.1, affine=True, track_running_stats=True) (conv2): Conv2d(256, 256,
kernel_size=(3, 3), stride=(1, 1), padding=(1, 1), bias=False) (bn2):
BatchNorm2d(256, eps=1e-05, momentum=0.1, affine=True, track_running_stats=True)

```



```

(conv3): Conv2d(256, 1024, kernel_size=(1, 1), stride=(1, 1), bias=False) (bn3):
BatchNorm2d(1024, eps=1e-05, momentum=0.1, affine=True, track_running_stats=True)
(rel): ReLU(inplace=True) ) (4): Bottleneck( (conv1): Conv2d(1024, 256,
kernel_size=(1, 1), stride=(1, 1), bias=False) (bn1): BatchNorm2d(256, eps=1e-05,
momentum=0.1, affine=True, track_running_stats=True) (conv2): Conv2d(256, 256,
kernel_size=(3, 3), stride=(1, 1), padding=(1, 1), bias=False) (bn2):
BatchNorm2d(256, eps=1e-05, momentum=0.1, affine=True, track_running_stats=True)
(conv3): Conv2d(256, 1024, kernel_size=(1, 1), stride=(1, 1), bias=False) (bn3):
BatchNorm2d(1024, eps=1e-05, momentum=0.1, affine=True, track_running_stats=True)
(rel): ReLU(inplace=True) ) (5): Bottleneck( (conv1): Conv2d(1024, 256,
kernel_size=(1, 1), stride=(1, 1), bias=False) (bn1): BatchNorm2d(256, eps=1e-05,
momentum=0.1, affine=True, track_running_stats=True) (conv2): Conv2d(256, 256,
kernel_size=(3, 3), stride=(1, 1), padding=(1, 1), bias=False) (bn2):
BatchNorm2d(256, eps=1e-05, momentum=0.1, affine=True, track_running_stats=True)
(conv3): Conv2d(256, 1024, kernel_size=(1, 1), stride=(1, 1), bias=False) (bn3):
BatchNorm2d(1024, eps=1e-05, momentum=0.1, affine=True, track_running_stats=True)
(rel): ReLU(inplace=True) ) ) (layer4): Sequential( (0): Bottleneck( (conv1):
Conv2d(1024, 512, kernel_size=(1, 1), stride=(1, 1), bias=False) (bn1):
BatchNorm2d(512, eps=1e-05, momentum=0.1, affine=True, track_running_stats=True)
(conv2): Conv2d(512, 512, kernel_size=(3, 3), stride=(2, 2), padding=(1, 1),
bias=False) (bn2): BatchNorm2d(512, eps=1e-05, momentum=0.1, affine=True,
track_running_stats=True) (conv3): Conv2d(512, 2048, kernel_size=(1, 1), stride=(1,
1), bias=False) (bn3): BatchNorm2d(2048, eps=1e-05, momentum=0.1, affine=True,
track_running_stats=True) (relu): ReLU(inplace=True) (downsample): Sequential( (0):
Conv2d(1024, 2048, kernel_size=(1, 1), stride=(2, 2), bias=False) (1):
BatchNorm2d(2048, eps=1e-05, momentum=0.1, affine=True, track_running_stats=True) )
(1): Bottleneck( (conv1): Conv2d(2048, 512, kernel_size=(1, 1), stride=(1, 1),
bias=False) (bn1): BatchNorm2d(512, eps=1e-05, momentum=0.1, affine=True,
track_running_stats=True) (conv2): Conv2d(512, 512, kernel_size=(3, 3), stride=(1,
1), padding=(1, 1), bias=False) (bn2): BatchNorm2d(512, eps=1e-05, momentum=0.1,
affine=True, track_running_stats=True) (conv3): Conv2d(512, 2048, kernel_size=(1,
1), stride=(1, 1), bias=False) (bn3): BatchNorm2d(2048, eps=1e-05, momentum=0.1,
affine=True, track_running_stats=True) (relu): ReLU(inplace=True) ) (2): Bottleneck(
(conv1): Conv2d(2048, 512, kernel_size=(1, 1), stride=(1, 1), bias=False) (bn1):
BatchNorm2d(512, eps=1e-05, momentum=0.1, affine=True, track_running_stats=True)
(conv2): Conv2d(512, 512, kernel_size=(3, 3), stride=(1, 1), padding=(1, 1),
bias=False) (bn2): BatchNorm2d(512, eps=1e-05, momentum=0.1, affine=True,
track_running_stats=True) (conv3): Conv2d(512, 2048, kernel_size=(1, 1), stride=(1,

```

```
1), bias=False) (bn3): BatchNorm2d(2048, eps=1e-05, momentum=0.1, affine=True,
track_running_stats=True) (relu): ReLU(inplace=True) ) ) (avgpool):
AdaptiveAvgPool2d(output_size=(1, 1)) (fc): Linear(in_features=2048,
out_features=1000, bias=True) )
```

ResNet50 Config를 사용한 ResNet 모델 사용

```
OUTPUT_DIM = 2 # 두 개의 클래스(개, 고양이) 사용
model = ResNet(resnet50_config, OUTPUT_DIM)
print(model)
```

```
ResNet( (conv1): Conv2d(3, 64, kernel_size=(7, 7), stride=(2, 2), padding=(3, 3),
bias=False) (bn1): BatchNorm2d(64, eps=1e-05, momentum=0.1, affine=True,
track_running_stats=True) (relu): ReLU(inplace=True) (maxpool):
MaxPool2d(kernel_size=3, stride=2, padding=1, dilation=1, ceil_mode=False) (layer1):
Sequential( (0): Bottleneck( (conv1): Conv2d(64, 64, kernel_size=(1, 1), stride=(1,
1), bias=False) (bn1): BatchNorm2d(64, eps=1e-05, momentum=0.1, affine=True,
track_running_stats=True) (conv2): Conv2d(64, 64, kernel_size=(3, 3), stride=(1, 1),
padding=(1, 1), bias=False) (bn2): BatchNorm2d(64, eps=1e-05, momentum=0.1,
affine=True, track_running_stats=True) (conv3): Conv2d(64, 256, kernel_size=(1, 1),
stride=(1, 1), bias=False) (bn3): BatchNorm2d(256, eps=1e-05, momentum=0.1,
affine=True, track_running_stats=True) (relu): ReLU(inplace=True) (downsample):
Sequential( (0): Conv2d(64, 256, kernel_size=(1, 1), stride=(1, 1), bias=False) (1):
BatchNorm2d(256, eps=1e-05, momentum=0.1, affine=True, track_running_stats=True) ) )
(1): Bottleneck( (conv1): Conv2d(256, 64, kernel_size=(1, 1), stride=(1, 1),
bias=False) (bn1): BatchNorm2d(64, eps=1e-05, momentum=0.1, affine=True,
track_running_stats=True) (conv2): Conv2d(64, 64, kernel_size=(3, 3), stride=(1, 1),
padding=(1, 1), bias=False) (bn2): BatchNorm2d(64, eps=1e-05, momentum=0.1,
affine=True, track_running_stats=True) (conv3): Conv2d(64, 256, kernel_size=(1, 1),
stride=(1, 1), bias=False) (bn3): BatchNorm2d(256, eps=1e-05, momentum=0.1,
affine=True, track_running_stats=True) (relu): ReLU(inplace=True) ) ) (layer2):
Sequential( (0): Bottleneck( (conv1): Conv2d(256, 128, kernel_size=(1, 1), stride=
(1, 1), bias=False) (bn1): BatchNorm2d(128, eps=1e-05, momentum=0.1, affine=True,
track_running_stats=True) (conv2): Conv2d(128, 128, kernel_size=(3, 3), stride=(2,
2), padding=(1, 1), bias=False) (bn2): BatchNorm2d(128, eps=1e-05, momentum=0.1,
affine=True, track_running_stats=True) (conv3): Conv2d(128, 512, kernel_size=(1, 1),
stride=(1, 1), bias=False) (bn3): BatchNorm2d(512, eps=1e-05, momentum=0.1,
affine=True, track_running_stats=True) (relu): ReLU(inplace=True) (downsample):
Sequential( (0): Conv2d(256, 512, kernel_size=(1, 1), stride=(2, 2), bias=False)
```

```

(1): BatchNorm2d(512, eps=1e-05, momentum=0.1, affine=True,
track_running_stats=True) ) ) (1): Bottleneck( (conv1): Conv2d(512, 128,
kernel_size=(1, 1), stride=(1, 1), bias=False) (bn1): BatchNorm2d(128, eps=1e-05,
momentum=0.1, affine=True, track_running_stats=True) (conv2): Conv2d(128, 128,
kernel_size=(3, 3), stride=(1, 1), padding=(1, 1), bias=False) (bn2):
BatchNorm2d(128, eps=1e-05, momentum=0.1, affine=True, track_running_stats=True)
(conv3): Conv2d(128, 512, kernel_size=(1, 1), stride=(1, 1), bias=False) (bn3):
BatchNorm2d(512, eps=1e-05, momentum=0.1, affine=True, track_running_stats=True)
(rel): ReLU(inplace=True) ) ) (layer3): Sequential( (0): Bottleneck( (conv1):
Conv2d(512, 256, kernel_size=(1, 1), stride=(1, 1), bias=False) (bn1):
BatchNorm2d(256, eps=1e-05, momentum=0.1, affine=True, track_running_stats=True)
(conv2): Conv2d(256, 256, kernel_size=(3, 3), stride=(2, 2), padding=(1, 1),
bias=False) (bn2): BatchNorm2d(256, eps=1e-05, momentum=0.1, affine=True,
track_running_stats=True) (conv3): Conv2d(256, 1024, kernel_size=(1, 1), stride=(1,
1), bias=False) (bn3): BatchNorm2d(1024, eps=1e-05, momentum=0.1, affine=True,
track_running_stats=True) (relu): ReLU(inplace=True) (downsample): Sequential( (0):
Conv2d(512, 1024, kernel_size=(1, 1), stride=(2, 2), bias=False) (1):
BatchNorm2d(1024, eps=1e-05, momentum=0.1, affine=True, track_running_stats=True) )
) (1): Bottleneck( (conv1): Conv2d(1024, 256, kernel_size=(1, 1), stride=(1, 1),
bias=False) (bn1): BatchNorm2d(256, eps=1e-05, momentum=0.1, affine=True,
track_running_stats=True) (conv2): Conv2d(256, 256, kernel_size=(3, 3), stride=(1,
1), padding=(1, 1), bias=False) (bn2): BatchNorm2d(256, eps=1e-05, momentum=0.1,
affine=True, track_running_stats=True) (conv3): Conv2d(256, 1024, kernel_size=(1,
1), stride=(1, 1), bias=False) (bn3): BatchNorm2d(1024, eps=1e-05, momentum=0.1,
affine=True, track_running_stats=True) (relu): ReLU(inplace=True) ) ) (layer4):
Sequential( (0): Bottleneck( (conv1): Conv2d(1024, 512, kernel_size=(1, 1), stride=
(1, 1), bias=False) (bn1): BatchNorm2d(512, eps=1e-05, momentum=0.1, affine=True,
track_running_stats=True) (conv2): Conv2d(512, 512, kernel_size=(3, 3), stride=(2,
2), padding=(1, 1), bias=False) (bn2): BatchNorm2d(512, eps=1e-05, momentum=0.1,
affine=True, track_running_stats=True) (conv3): Conv2d(512, 2048, kernel_size=(1,
1), stride=(1, 1), bias=False) (bn3): BatchNorm2d(2048, eps=1e-05, momentum=0.1,
affine=True, track_running_stats=True) (relu): ReLU(inplace=True) (downsample):
Sequential( (0): Conv2d(1024, 2048, kernel_size=(1, 1), stride=(2, 2), bias=False)
(1): BatchNorm2d(2048, eps=1e-05, momentum=0.1, affine=True,
track_running_stats=True) ) ) (1): Bottleneck( (conv1): Conv2d(2048, 512,
kernel_size=(1, 1), stride=(1, 1), bias=False) (bn1): BatchNorm2d(512, eps=1e-05,
momentum=0.1, affine=True, track_running_stats=True) (conv2): Conv2d(512, 512,
kernel_size=(3, 3), stride=(1, 1), padding=(1, 1), bias=False) (bn2):

```

```
BatchNorm2d(512, eps=1e-05, momentum=0.1, affine=True, track_running_stats=True)
(conv3): Conv2d(512, 2048, kernel_size=(1, 1), stride=(1, 1), bias=False) (bn3):
BatchNorm2d(2048, eps=1e-05, momentum=0.1, affine=True, track_running_stats=True)
(relu): ReLU(inplace=True) ) ) (avgpool): AdaptiveAvgPool2d(output_size=(1, 1))
(fc): Linear(in_features=2048, out_features=2, bias=True) )
```

직접 정의한 네트워크와 사전 정의된 네트워크가 다르지 않다. 앞으로 ResNet을 사용할 필요가 있을 경우 사전 훈련된 모델을 코드 한두 줄로 불러와 사용하게 될 것이다.

옵티마이저와 손실 함수 정의

```
optimizer = optim.Adam(model.parameters(), lr=1e-7)
criterion = nn.CrossEntropyLoss()

model = model.to(device)
criterion = criterion.to(device)
```

모델 학습 정확도 측정 함수 정의

```
def calculate_topk_accuracy(y_pred, y, k=2):
    with torch.no_grad():
        batch_size = y.shape[0]
        _, top_pred = y_pred.topk(k, 1) # ①
        top_pred = top_pred.t()
        correct = top_pred.eq(y.view(1, -1).expand_as(top_pred)) ②
        correct_1 = correct[:1].reshape(-1).float().sum(0, keepdim=True)
        correct_k = correct[:k].reshape(-1).float().sum(0, keepdim=True) # 이미지의 정확한 레이블 부여
        acc_1 = correct_1 / batch_size
        acc_k = correct_k / batch_size
    return acc_1, acc_k
```

① `tensor.topk`는 `torch.argmax`와 같은 효과이다. 주어진 텐서에서 가장 큰 값의 인덱스를 얻기 위해 사용한다. 즉, 네트워크 출력에서 가장 확률이 높은 값의 인덱스를 반환한다.

② `t()`는 차원 0과 1을 전치하겠다는 의미이다.

③ 텐서를 비교하는 함수로, 텐서가 서로 같은지를 비교한다면 `torch.eq`, 다른지를 비교한다면 `torch.ne`, 크거나 같은지를 비교한다면 `torch.ge`를 사용한다.

모델 학습 함수 정의

```
def train(model, iterator, optimizer, criterion, scheduler, device):
    epoch_loss = 0
    epoch_acc_1 = 0
    epoch_acc_5 = 0
    model.train()

    for (x, y) in iterator:
```

```

x = x.to(device)
y = y.to(device)

optimizer.zero_grad()
y_pred = model(x)
loss = criterion(y_pred[0], y)

acc_1, acc_5 = calculate_topk_accuracy(y_pred[0], y)
loss.backward()
optimizer.step()

epoch_loss += loss.item()
epoch_acc_1 += acc_1.item() # 모델이 첫 번째로 예측한 레이블이 붙여짐
epoch_acc_5 += acc_5.item() # 이미지에 정확한 레이블이 붙여질 것이기에 정확도가 100%일 것임

epoch_loss /= len(iterator)
epoch_acc_1 /= len(iterator)
epoch_acc_5 /= len(iterator)
return epoch_loss, epoch_acc_1, epoch_acc_5

```

모델 평가 함수 정의

```

def evaluate(model, iterator, criterion, device):
    epoch_loss = 0
    epoch_acc_1 = 0
    epoch_acc_5 = 0

    model.eval()
    with torch.no_grad():
        for (x,y) in iterator:
            x = x.to(device)
            y = y.to(device)
            y_pred = model(x)
            loss = criterion(y_pred[0], y)

            acc_1, acc_5 = calculate_topk_accuracy(y_pred[0], y)
            epoch_loss += loss.item()
            epoch_acc_1 += acc_1.item()
            epoch_acc_5 += acc_5.item()

    epoch_loss /= len(iterator)
    epoch_acc_1 /= len(iterator)
    epoch_acc_5 /= len(iterator)
    return epoch_loss, epoch_acc_1, epoch_acc_5

```

모델 학습 시간 측정 함수 정의

```

def epoch_time(start_time, end_time):
    elapsed_time = end_time - start_time
    elapsed_mins = int(elapsed_time / 60)
    elapsed_secs = int(elapsed_time - (elapsed_mins * 60))
    return elapsed_mins, elapsed_secs

```

모델 학습

```
best_valid_loss = float('inf')
EPOCHS = 10

scheduler = torch.optim.lr_scheduler.StepLR(optimizer, step_size=5, gamma=0.1)

for epoch in range(EPOCHS):
    start_time = time.monotonic()

    train_loss , train_acc_1, train_acc_5 = train(model, train_iterator, optimizer,
                                                  criterion, scheduler, device)

    valid_loss, valid_acc_1, valid_acc_5 = evaluate(model, valid_iterator, criterion,
                                                    device)

    scheduler.step()

    if valid_loss < best_valid_loss:
        best_valid_loss = valid_loss
        torch.save(model.state_dict(), "/content/drive/MyDrive/data/ResNet-model.pt")

    end_time = time.monotonic()
    epoch_mins, epoch_secs = epoch_time(start_time, end_time)

    print(f'Epoch: {epoch+1:02} | Epoch Time: {epoch_mins}m {epoch_secs}s')
    print(f'\tTrain Loss: {train_loss:.3f} | Train Acc @1: {train_acc_1*100:6.2f}% | ' \
          f'\tTrain Acc @5: {train_acc_5*100:6.2f}%')
    print(f'\tValid Loss: {valid_loss:.3f} | Valid Acc @1: {valid_acc_1*100:6.2f}% | ' \
          f'\tValid Acc @5: {valid_acc_5*100:6.2f}%')
```

```
Epoch: 01 | Epoch Time: 0m 5s Train Loss: 0.712 | Train Acc @1: 49.76% | Train Acc
@5: 100.00% Valid Loss: 0.699 | Valid Acc @1: 51.19% | Valid Acc @5: 100.00% Epoch:
02 | Epoch Time: 0m 5s Train Loss: 0.715 | Train Acc @1: 49.28% | Train Acc @5:
100.00% Valid Loss: 0.708 | Valid Acc @1: 51.19% | Valid Acc @5: 100.00% Epoch: 03 |
Epoch Time: 0m 5s Train Loss: 0.710 | Train Acc @1: 49.52% | Train Acc @5: 100.00%
Valid Loss: 0.714 | Valid Acc @1: 51.19% | Valid Acc @5: 100.00% Epoch: 04 | Epoch
Time: 0m 5s Train Loss: 0.712 | Train Acc @1: 50.72% | Train Acc @5: 100.00% Valid
Loss: 0.708 | Valid Acc @1: 50.15% | Valid Acc @5: 100.00% Epoch: 05 | Epoch Time:
0m 5s Train Loss: 0.702 | Train Acc @1: 50.72% | Train Acc @5: 100.00% Valid Loss:
0.699 | Valid Acc @1: 50.15% | Valid Acc @5: 100.00% Epoch: 06 | Epoch Time: 0m 5s
Train Loss: 0.709 | Train Acc @1: 49.28% | Train Acc @5: 100.00% Valid Loss: 0.699 |
Valid Acc @1: 50.15% | Valid Acc @5: 100.00% Epoch: 07 | Epoch Time: 0m 5s Train
Loss: 0.700 | Train Acc @1: 50.00% | Train Acc @5: 100.00% Valid Loss: 0.695 | Valid
Acc @1: 49.11% | Valid Acc @5: 100.00% Epoch: 08 | Epoch Time: 0m 5s Train Loss:
0.705 | Train Acc @1: 49.76% | Train Acc @5: 100.00% Valid Loss: 0.693 | Valid Acc
@1: 49.11% | Valid Acc @5: 100.00% Epoch: 09 | Epoch Time: 0m 5s Train Loss: 0.698 |
```

Train Acc @1: 51.44% | Train Acc @5: 100.00% Valid Loss: 0.693 | Valid Acc @1:
50.15% | Valid Acc @5: 100.00% Epoch: 10 | Epoch Time: 0m 5s Train Loss: 0.707 |
Train Acc @1: 50.00% | Train Acc @5: 100.00% Valid Loss: 0.691 | Valid Acc @1:
49.11% | Valid Acc @5: 100.00%

테스트 데이터셋을 이용한 모델 예측

```
import pandas as pd
id_list = []
pred_list = []
_id = 0
with torch.no_grad():
    for test_path in test_images_filepaths:
        img = Image.open(test_path)
        _id = test_path.split('/')[-1].split('.')[1]
        transform = ImageTransform(size, mean, std)
        img = transform(img, phase='val')
        img = img.unsqueeze(0)
        img = img.to(device)

        model.eval()
        outputs = model(img)
        preds = F.softmax(outputs[0], dim=1)[: , 1].tolist()
        id_list.append(_id)
        pred_list.append(preds[0])

res = pd.DataFrame({
    'id': id_list,
    'label': pred_list
})

res.sort_values(by='id', inplace=True)
res.reset_index(drop=True, inplace=True)

res.to_csv('/content/drive/MyDrive/data/ResNet.csv', index=False)
res.head(10)
```

	id	label
0	109	0.560423
1	145	0.546428
2	15	0.605402
3	162	0.614576
4	167	0.568054
5	200	0.644561
6	210	0.605344
7	211	0.553360
8	213	0.557236
9	224	0.526934

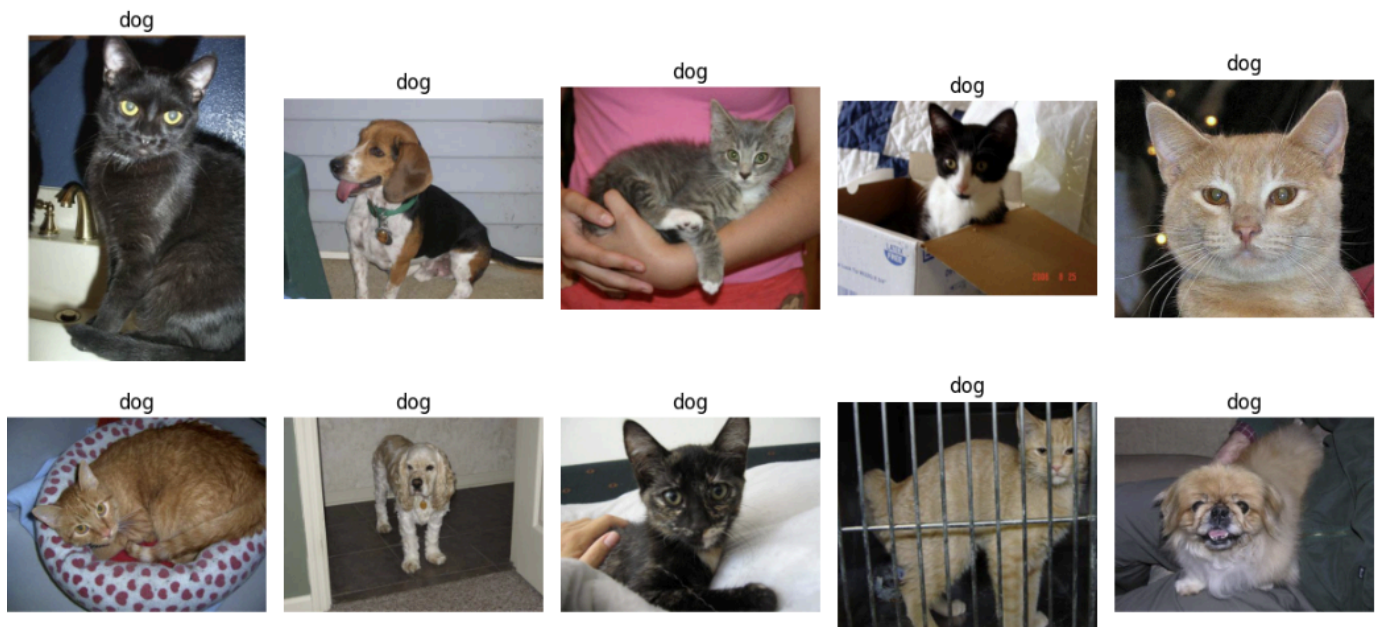
레이블이 0.5보다 크면 개, 0.5보다 작으면 고양이를 의미한다.

모델 예측에 대한 결과 출력

```
class_ = classes = {0:'cat', 1:'dog'}
def display_image_grid(images_filepaths, predicted_labels=(), cols=5):
    rows = len(images_filepaths) // cols
    figure, ax = plt.subplots(nrows=rows, ncols=cols, figsize=(12,6))
    for i, images_filepath in enumerate(images_filepaths):
        image = cv2.imread(images_filepath)
        image = cv2.cvtColor(image, cv2.COLOR_BGR2RGB)

        a = random.choice(res['id'].values)
        label = res.loc[res['id'] == a, 'label'].values[0]

        if label > 0.5:
            label = 1
        else:
            label = 0
        ax.ravel()[i].imshow(image)
        ax.ravel()[i].set_title(class_[label])
        ax.ravel()[i].set_axis_off()
    plt.tight_layout()
    plt.show()
display_image_grid(test_images_filepaths)
```

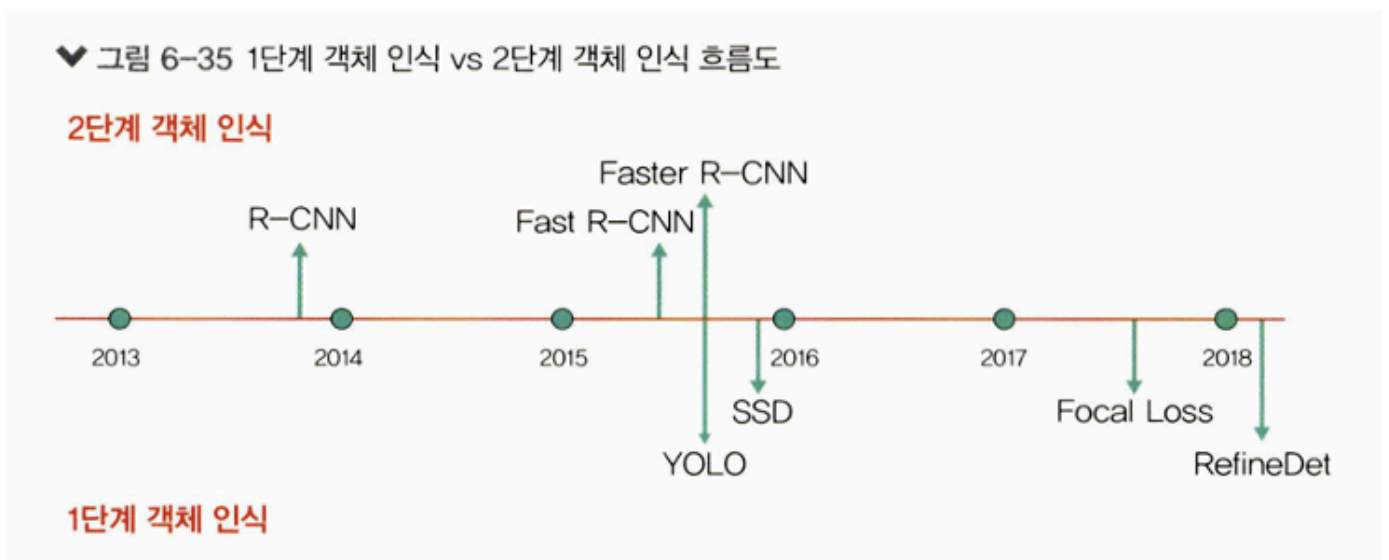
신경망은 이미 구현된 모델을 재사용할 수 있는 것이 많기 때문에 누군가가 만들어 놓은 신경망을 가져다 쓰기만 하면 된다. 중요한 점은 내가 가진 데이터에 가장 적합한 모델을 선택하는 것이다. 이후에는 앞서 배운 전이 학습을 사용해 약간의 튜닝만 진행하면 된다.

6.2 객체 인식을 위한 신경망

객체 인식: 이미지나 영상 내에 있는 객체를 식별하는 컴퓨터 비전 기술

즉, 이미지나 영상 내에 있는 여러 객체에 대해 각 객체가 무엇인지 분류하는 문제와 그 객체 위치가 어디인지 박스로 나타내는 위치 검출 문제를 다루는 분야이다.

딥러닝을 이용한 객체 인식 알고리즘은 1단계 객체 인식과 2단계 객체 인식으로 나눌 수 있다.



1단계 객체 인식은 분류와 위치 검출을 동시에 행하는 방법이고, 2단계 객체 인식은 두 문제를 순차적으로 행하는 방법이다. 1단계 객체 인식은 비교적 빠르지만 정확도가 낮고, 2단계 객체 인식은 비교적 느리지만 정확도가 높다.

여기에서는 2단계 객체 인식 알고리즘을 알아볼 것이다.

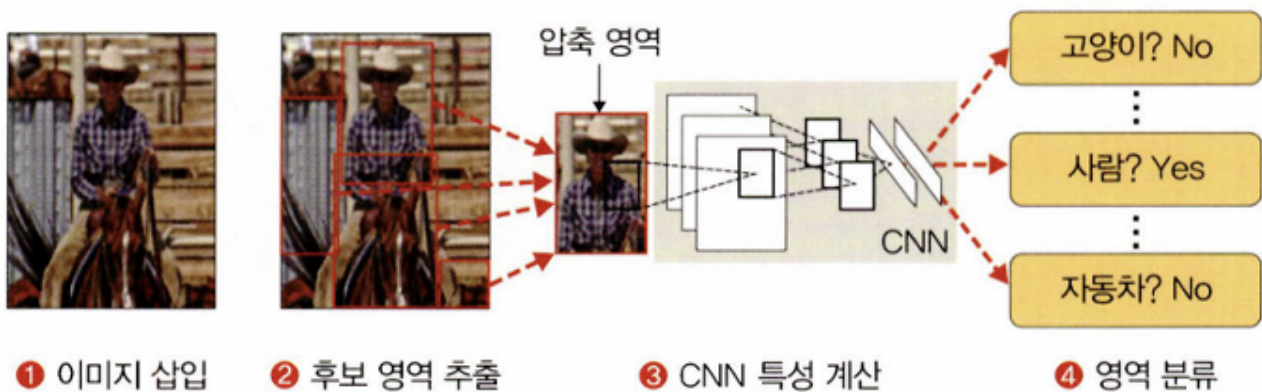
6.2.1 R-CNN

예전의 객체 인식 알고리즘들은 슬라이딩 윈도우 방식, 즉 일정한 크기를 가지는 윈도우를 가지고 이미지의 모든 영역을 탐색하면서 객체를 검출해 내는 방식이었다. 하지만 알고리즘의 비효율성 때문에 많이 사용하지 않으며, 현재는 선택적 탐색 알고리즘을 적용한 후보 영역을 많이 사용한다.

R-CNN은 이미지 분류를 수행하는 CNN과 이미지에서 객체가 있을 만한 영역을 제안해 주는 후보 영역 알고리즘을 결합한 알고리즘이다.

수행 과정은 다음과 같다.

▼ 그림 6-36 R-CNN 학습 절차



1. 이미지를 입력으로 받는다.
2. 2000개의 바운딩 박스를 선택적 탐색 알고리즘으로 추출한 후 잘라 내고, CNN 모델에 넣기 위해 같은 크기(227x227 픽셀)로 통일한다.
3. 크기가 동일한 이미지 2000개에 각각 CNN 모델을 적용한다.
4. 각각 분류를 진행해 결과를 도출한다.

선택적 탐색

선택적 탐색은 객체 인식이나 검출을 위한 가능한 후보 영역(객체가 있을 만한 위치, 영역)을

알아내는 방법이다. 선택적 탐색은 분할 방식을 이용하여 seed를 선정하고, 그 시드에 대한 완전 탐색을 적용한다.

선택적 탐색은 다음 세 단계 과정을 거친다.

1단계. 초기 영역 생성

입력된 이미지를 영역 다수 개로 분할한다.



입력 이미지



분할



후보 영역

2단계. 작은 영역의 통합

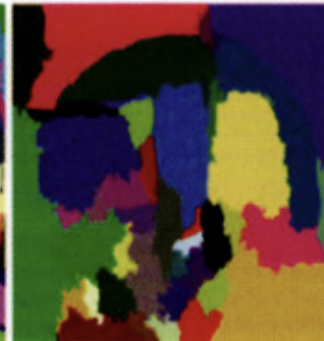
1단계에서 영역 여러 개로 나눈 것들을 비슷한 영역끼리 통합하는데, 이때 그리디 알고리즘을 사용하여 비슷한 영역이 하나로 통합될 때까지 반복한다.



입력 이미지



초기 분할



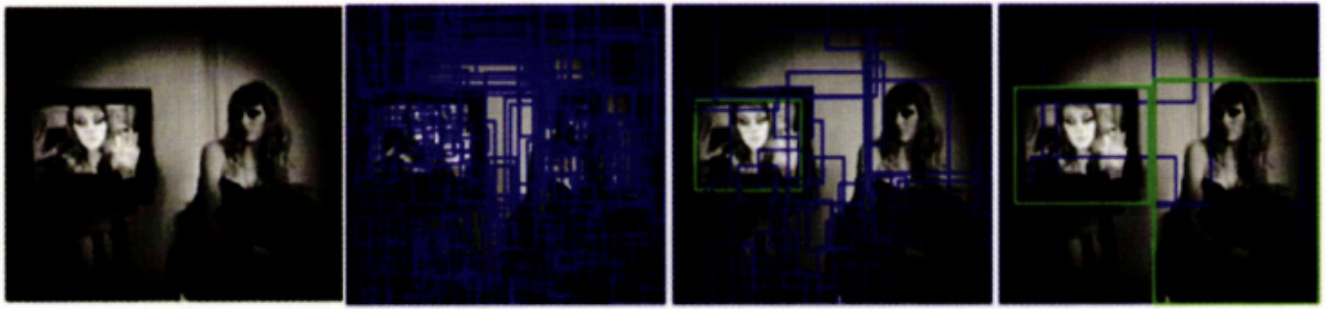
비슷한 영역 통합



비슷한 영역 통합

3단계. 후보 영역 생성

2단계에서 통합된 이미지들을 기반으로 다음 그림과 같이 후보 영역을 추출한다.



입력 이미지

비슷한 영역을 통합하여 후보 영역 생성

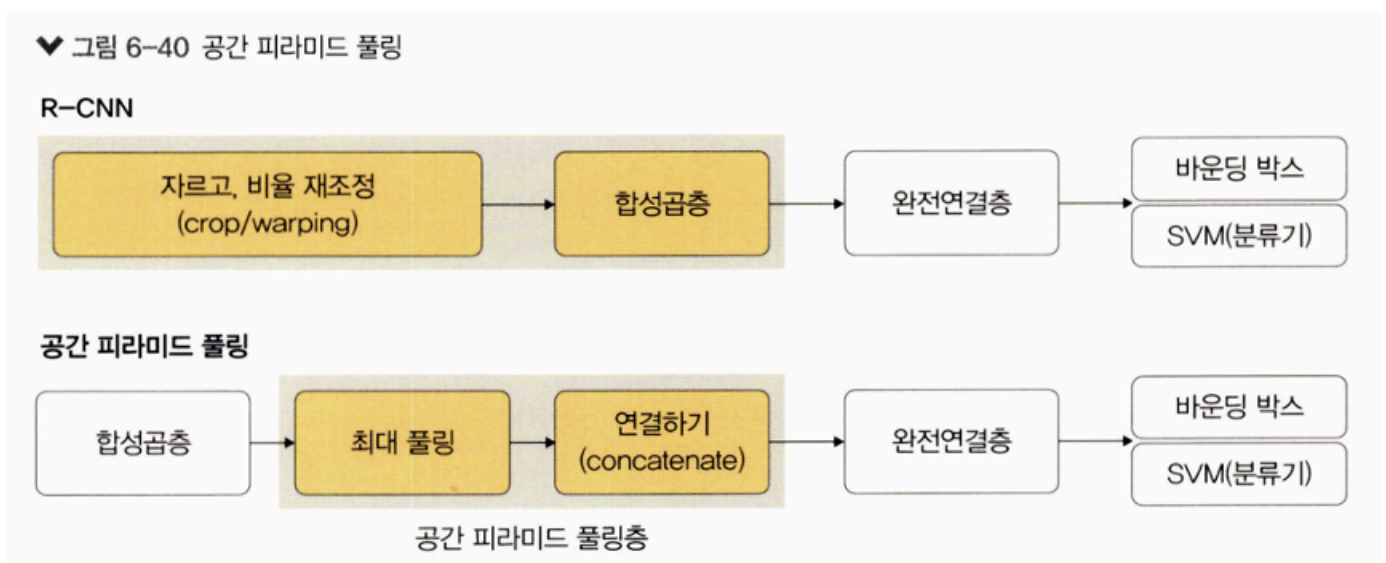
R-CNN은 성능이 뛰어나기는 하지만 다음과 같은 단점으로 크게 발전하지는 못했다.

1. 세 단계의 복잡한 학습 과정
2. 긴 학습 시간과 대용량 저장 공간
3. 객체 검출 속도 문제

이러한 문제를 해결하기 위해 Fast R-CNN이 생겼다.

6.2.2 공간 피라미드 풀링

기존 CNN 구조들은 모두 완전연결층을 위해 입력 이미지를 고정해야 했다. 그렇기 때문에 신경망을 통과시키려면 이미지를 고정된 크기로 자르거나 비율을 조정해야 했다. 하지만 이렇게 하면 물체의 일부분이 잘리거나 본래의 생김새와 달라지는 문제점이 있다. 이러한 문제를 해결하고자 공간 피라미드 풀링을 도입했다.



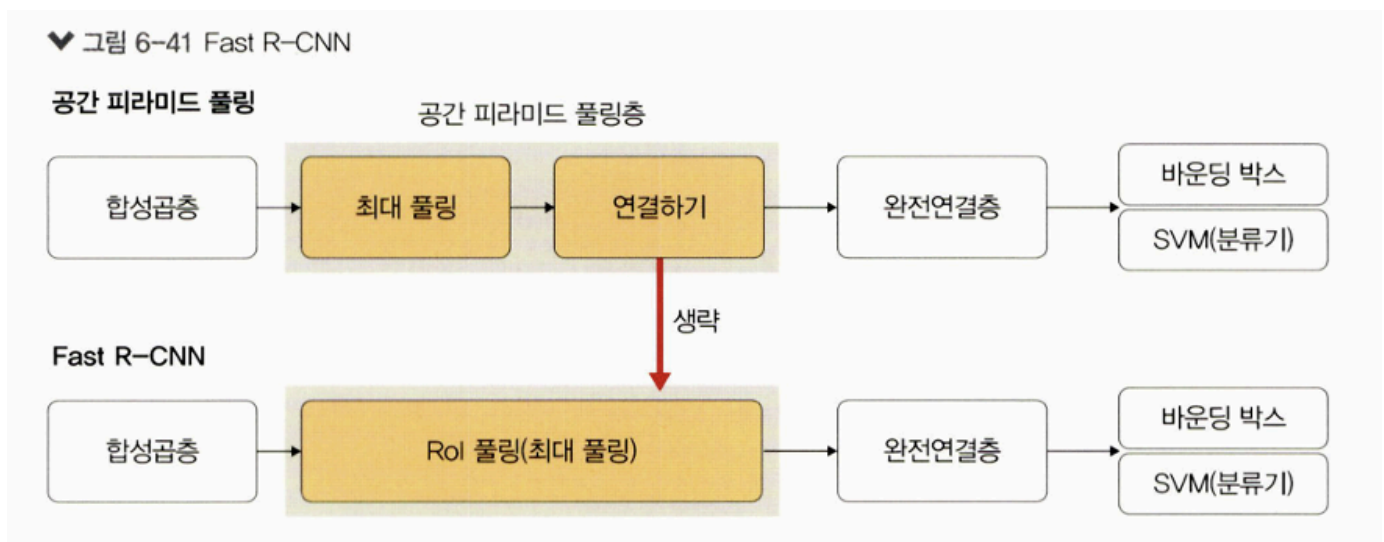
공간 피라미드 풀링은 입력 이미지의 크기에 관계없이 합성곱층을 통과시키고, 완전연결층에

전달되기 전에 특성 맵들을 동일한 크기로 조절해 주는 풀링층을 적용하는 기법이다.

입력 이미지의 크기를 조절하지 않고 합성곱층을 통과시키기 때문에 원본 이미지의 특징이 훼손되지 않는 특성 맵을 얻을 수 있다. 또한, 이미지 분류나 객체 인식 같은 여러 작업에 적용할 수 있다는 장점이 있다.

6.2.3 Fast R-CNN

R-CNN은 긴 학습 시간이 문제였다. Fast R-CNN은 R-CNN의 속도 문제를 개선하기 위해 RoI 풀링을 도입했다. 즉, 선택적 탐색에서 찾은 바운딩 박스 정보가 CNN을 통과하면서 유지되도록 하고 최종 CNN 특성 맵은 풀링을 적용하여 완전연결층을 통과하도록 크기를 조정한다. 이렇게 하면 바운딩 박스마다 CNN을 돌리는 시간을 단축할 수 있다.



RoI 풀링

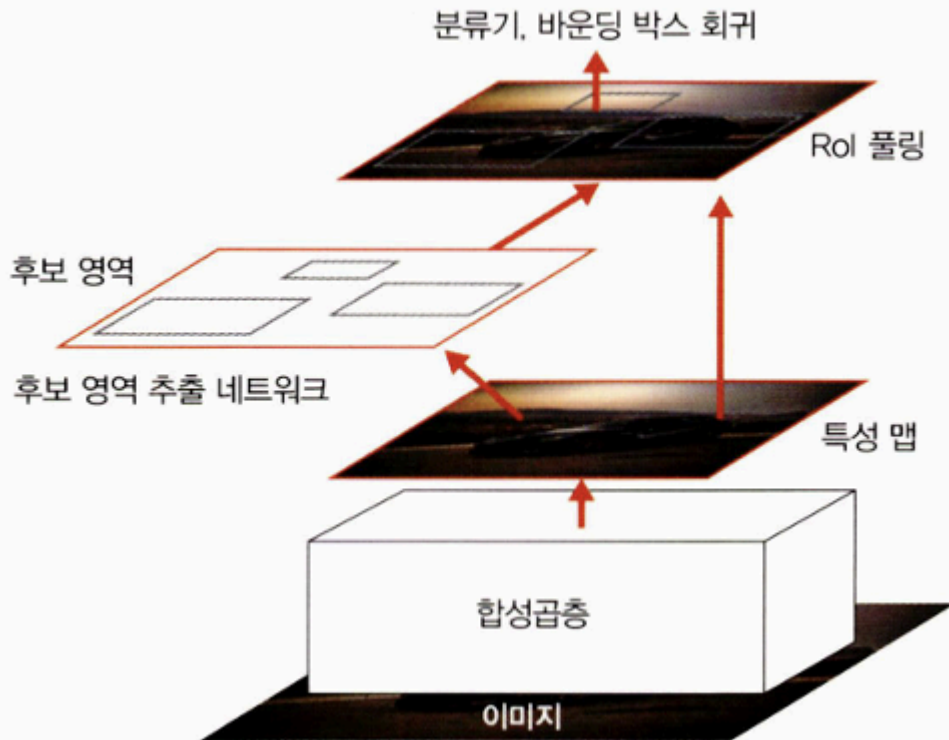
RoI 풀링은 크기가 다른 특성 맵의 영역마다 스트라이드를 다르게 최대 풀링을 적용하여 곱셈 곱하기를 동일하게 맞추는 방법이다.

6.2.4 Faster R-CNN

Faster R-CNN은 더욱 빠른 객체 인식을 수행하기 위한 네트워크이다. 기존 Fast R-CNN 속도의 걸림돌이었던 후보 영역 생성을 CNN 내부 네트워크에서 진행할 수 있도록 설계했다. 즉, Faster R-CNN은 기존 Fast R-CNN에 후보 영역 추출 네트워크를 추가한 것이 핵심이라고 할 수 있다. Faster R-CNN에서는 외부의 느린 선택적 탐색(CPU로 계산) 대신 내부의 빠른 RPN(GPU로 계산)을 사용한다.

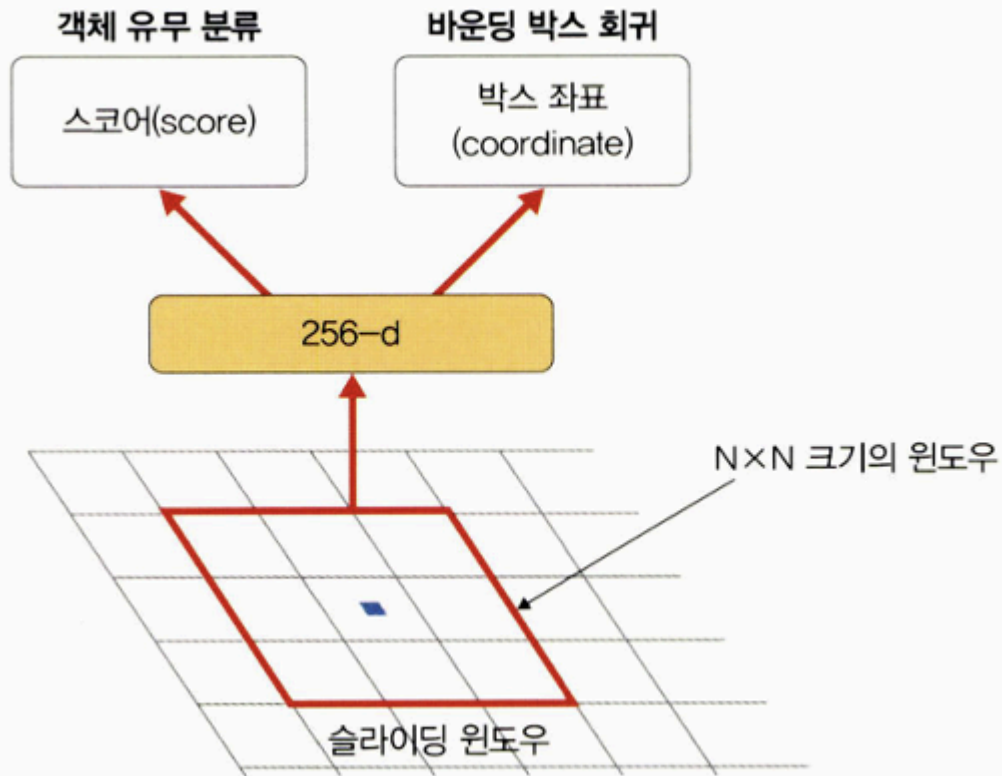
RPN은 다음 그림과 같이 마지막 합성곱층 다음에 위치하고, 그 뒤에 Fast R-CNN과 마찬가지로 RoI 풀링과 분류기, 바운딩 박스 회귀가 위치한다.

▼ 그림 6-43 Faster R-CNN



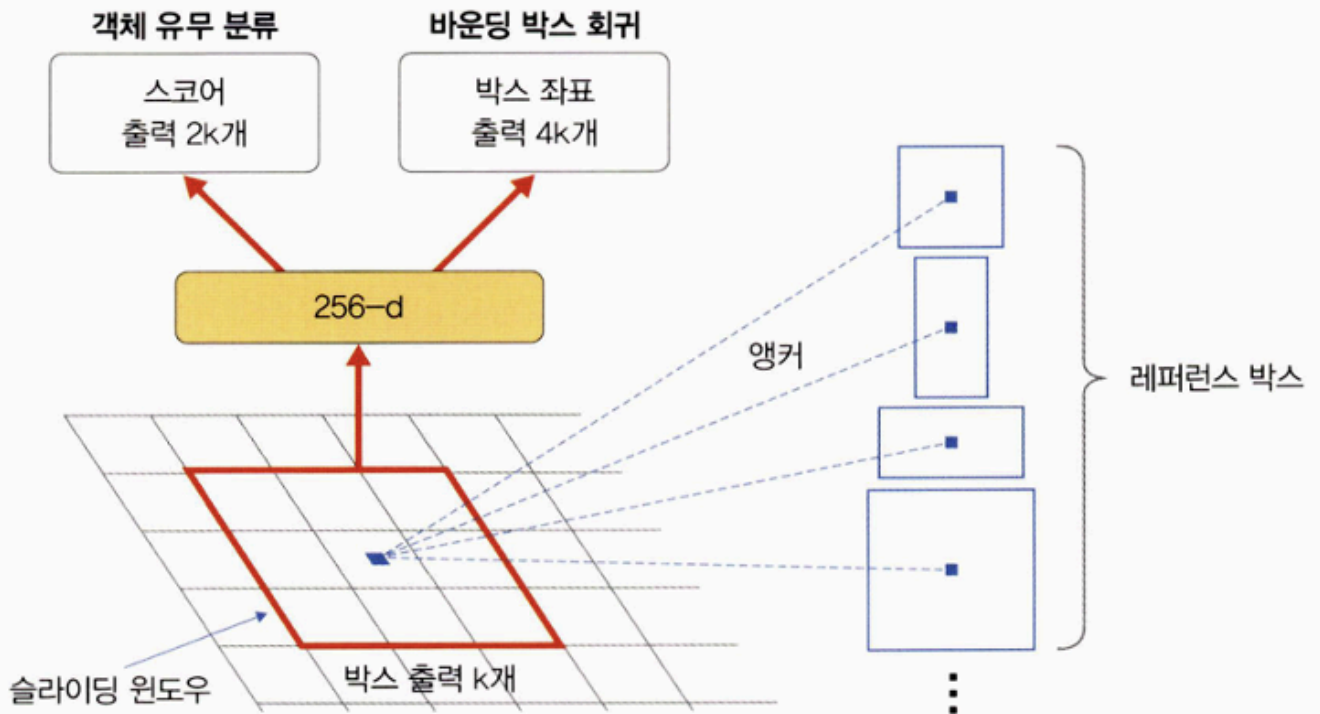
후보 영역 추출 네트워크는 특성 맵 $N \times N$ 크기의 작은 윈도우 영역을 입력으로 받고, 해당 영역에 객체의 존재 유무 판단을 위해 이진 분류를 수행하는 작은 네트워크를 생성한다. R-CNN, Fast R-CNN에서 사용되었던 바운딩 박스 회귀 또한 위치 보정(좌표점 추론)을 위해 추가한다. 또한, 하나의 특성 맵에서 모든 영역에 대한 객체의 존재 유무를 확인하기 위해서는 슬라이딩 윈도우 방식으로 앞서 설계한 작은 윈도우 영역($N \times N$ 크기)을 이용하여 객체를 탐색한다.

▼ 그림 6-44 후보 영역 추출 네트워크



하지만 후보 영역 추출 네트워크는 이미지에 존재하는 객체들의 크기와 비율이 다양하기 때문에 고정된 $N \times N$ 크기의 입력만으로 다양한 크기와 비율의 이미지를 수용하기 어려운 단점이 있다. 이러한 단점을 보완하기 위해 여러 크기와 비율의 레퍼런스 박스 k 개를 미리 정의하고 각각의 슬라이딩 윈도우 위치마다 박스 k 개를 출력하도록 설계하는데, 이 방식을 앵커라고 한다. 즉, 후보 영역 추출 네트워크의 출력 값은 모든 앵커 위치에 대해 각각 객체와 배경을 판단하는 $2k$ 개의 분류에 대한 출력과 x, y, w, h 위치 보정 값을 위한 $4k$ 개의 회귀 출력을 갖는다. 예를 들어 특성 맵 크기가 $w \times h$ 라면 하나의 특성 맵에 앵커가 총 $w \times h \times k$ 개 존재한다.

♥ 그림 6-45 앵커



6.3 이미지 분할을 위한 신경망

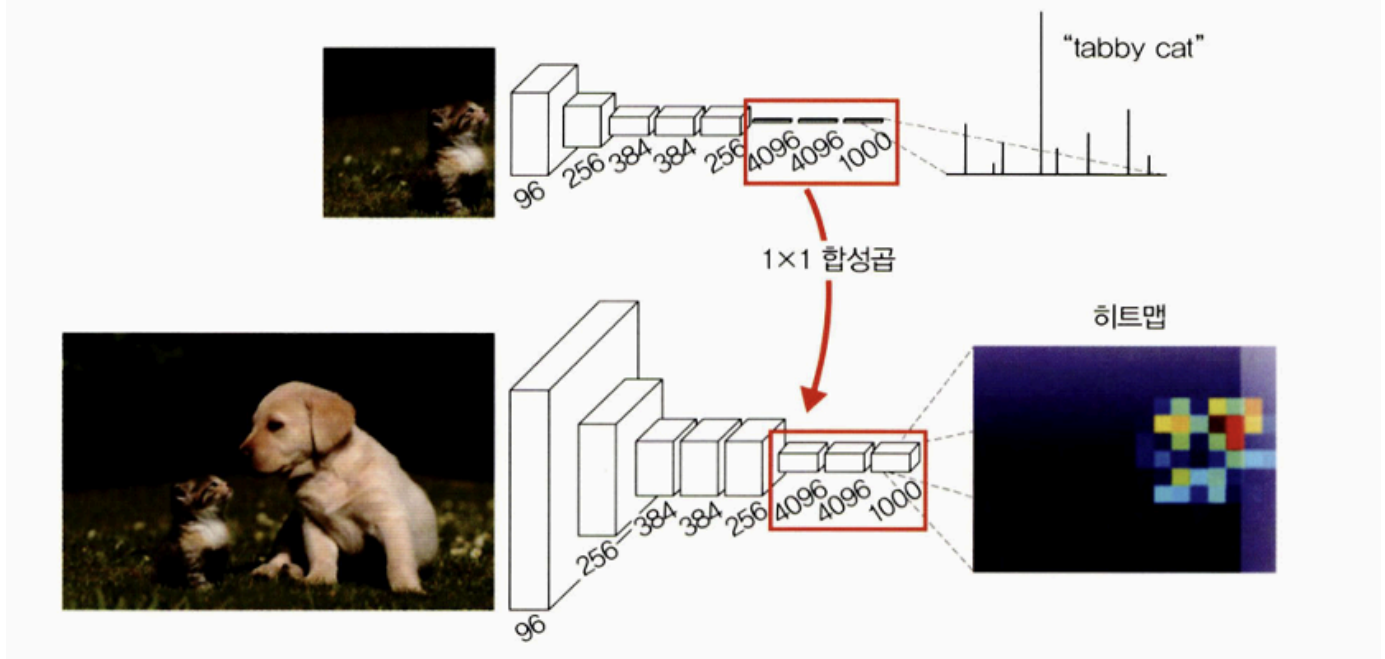
이미지 분할: 신경망을 훈련시켜 이미지를 픽셀 단위로 분할하는 것. 즉, 이미지를 픽셀 단위로 분할하여 이미지에 포함된 객체를 추출한다. 이미지 분할의 대표적 네트워크는 완전 합성곱 네트워크, 합성곱 & 역합성곱 네트워크, U-Net, PSPNet, DeepLabv3/DeepLabv3+가 있다.

6.3.1 완전 합성곱 네트워크

완전연결층의 한계는 고정된 크기의 입력만 받아들이며, 완전연결층을 거친 후에는 위치 정보가 사라진다는 것이다. 이러한 문제를 해결하기 위해 완전연결층을 1x1 합성곱으로 대체하는 것이 완전 합성곱 네트워크이다. 즉, 완전 합성곱 네트워크는 이미지 분류에서 우수한 성능을 보인 CNN 기반 모델(AlexNet, VGG16, GoogLeNet)을 변형시켜 이미지 분할에 적합하도록 만든 네트워크이다.

예를 들어 다음 그림과 같이 AlexNet 아래쪽에서 사용되었던 완전연결층 세 개를 1x1 합성곱으로 변환하면 위치 정보가 남아 있기 때문에 히트맵 그림과 같이 고양이의 취리를 확인할 수 있다.

▼ 그림 6-46 완전 합성곱 네트워크



또한 합성곱층으로 사용되기 때문에 입력 이미지에 대한 크기 제약이 사라진다는 장점이 있다.

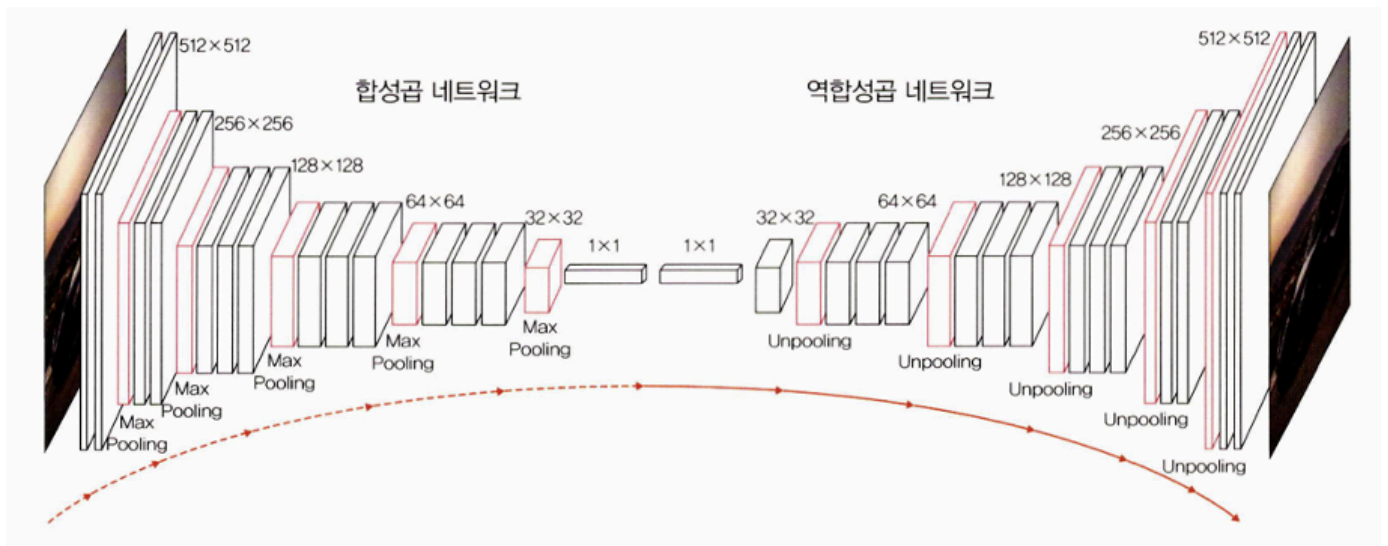
6.3.2 합성곱 & 역합성곱 네트워크

완전 합성곱 네트워크는 위치 정보가 보존된다는 장점에도 다음과 같은 단점이 있다.

- 여러 단계의 합성곱층과 풀링층을 거치면서 해상도가 낮아진다.
- 낮아진 해상도를 복원하기 위해 업샘플링 방식을 사용하기 때문에 이미지의 세부 정보들을 잃어버린다.

이러한 문제를 해결하기 위해 역합성곱 네트워크를 도입한 것이 합성곱 & 역합성곱 네트워크이다.

역합성곱은 CNN의 최종 출력 결과를 원래의 입력 이미지와 같은 크기로 만들고 싶을 때 사용한다. 시맨틱 분할 등에 활용할 수 있으며, 역합성곱을 업 샘플링이라고도 한다.



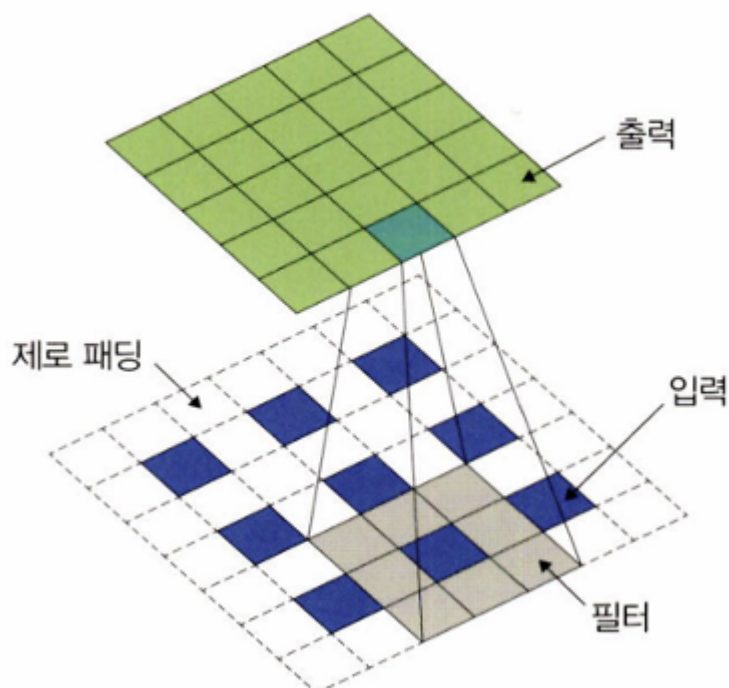
CNN에서 합성곱층은 합성곱을 사용하여 특성 맵 크기를 줄인다. 하지만 역합성곱은 이와 반대로 특성 맵 크기를 증가시키는 방식으로 동작한다.

역합성곱은 다음 방식으로 동작한다.

1. 각각의 픽셀 주위에 제로 패딩을 추가한다.
2. 이렇게 패딩된 것에 합성곱 연산을 수행한다.

아래 그림에서 파란색 픽셀이 입력이며, 초록색 픽셀이 출력이다. 이 파란색 픽셀 주위로 흰색 제로 패딩을 수행하고, 회색 필터로 합성곱 연산을 수행하면 초록색이 출력된다.

♥ 그림 6-48 역합성곱 진행 방식

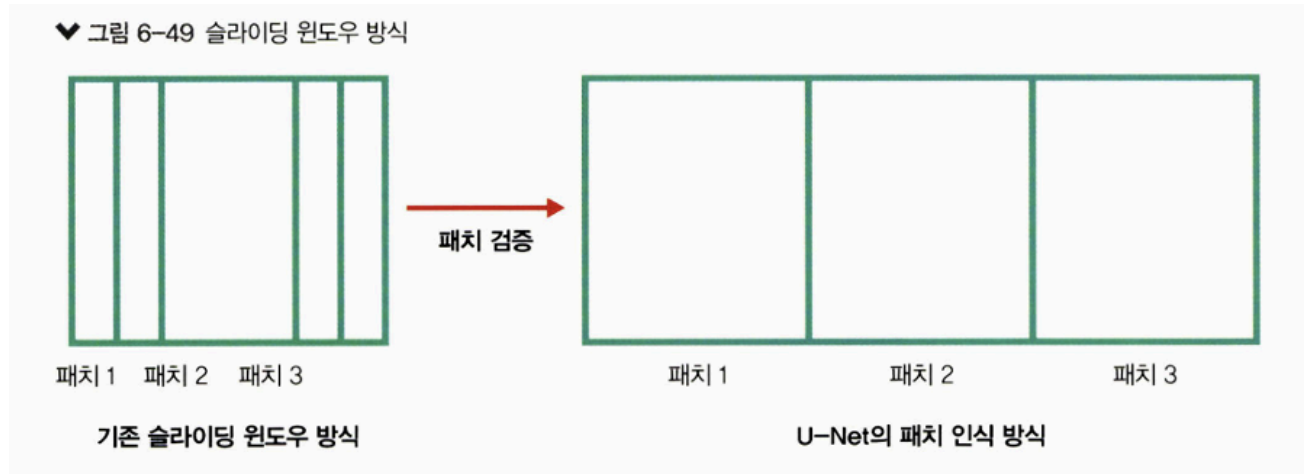


6.3.3 U-Net

U-Net: 바이오 메디컬 이미지 분할을 위한 합성곱 신경망이다. 메디컬 이미지의 분할과 관련해서 항상 회자되는 네트워크가 U-Net이다.

U-Net은 다음과 같은 특징이 있다.

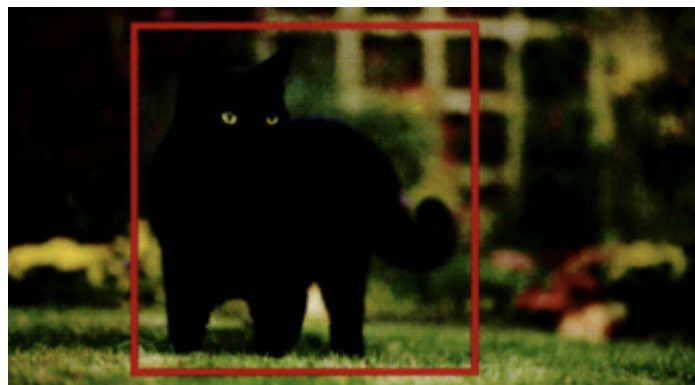
- 속도가 빠르다: 기존 슬라이딩 윈도우 방식은 이전 패치에서 검증이 끝난 부분을 다음 패치에서 또 검증하기 때문에 속도가 느리다. 하지만 U-Net은 이미 검증이 끝난 패치는 건너뛰기 때문에 속도가 빠르다.



- 트레이드오프에 빠지지 않는다: 일반적으로 패치 크기가 커진다면 넓은 범위의 이미지를 인식하는 데 뛰어나기 때문에 컨텍스트 인식에 탁월하다. 하지만 지역화에는 한계가 있다. 즉, 너무 많은 범위를 한 번에 인식하기 때문에 지역화에는 약하기 마련인데, U-Net은 컨텍스트 인식과 지역화 트레이드오프 문제를 개선했다.

지역화

지역화는 아래 그림과 같은 이미지 안에 객체(고양이) 위치 정보를 출력해 주는 것으로, 주로 바운딩 박스를 많이 사용한다. 바운딩 박스의 네 꼭짓점 픽셀 좌표가 출력되는 것이 아닌 왼쪽 위, 오른쪽 아래 좌표를 출력한다.

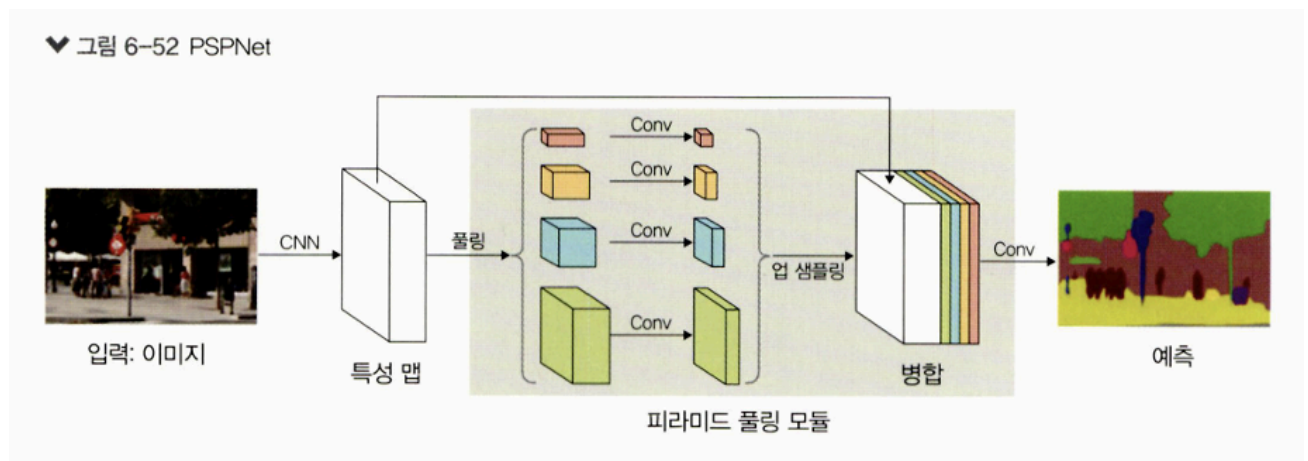


6.3.4 PSPNet

PSPNet은 CVPR 2017에서 발표된 시멘틱 분할 알고리즘이다.

PSPNet 역시 완전연결층의 한계를 극복하기 위해 피라미드 풀링 모듈을 추가했으며 훈련 과정은 다음과 같다.

1. 이미지 출력이 서로 다른 크기가 되도록 여러 차례 풀링을 한다. 즉, 1x1, 2x2, 3x3, 6x6 크기로 풀링을 수행하는데, 이때 1x1 크기의 특성 맵은 가장 광범위한 정보를 담는다. 각각 다른 크기의 특성 맵은 서로 다른 영역들의 정보를 담는다.
2. 이후 1x1 합성곱을 사용해 채널 수를 조정한다. 풀링층 개수를 N이라고 할 때 출력 채널 수 = 입력 채널 수 / N이 된다.
3. 이후 모듈의 입력 크기에 맞게 특성 맵을 업 샘플링한다. 이 과정에서 양선형 보간법이 사용된다.
4. 원래 특성 맵과 1~3 과정에서 생성한 새로운 특성 맵들을 병합한다.



구현에 따라 풀링을 다르게 설정할 수 있다.

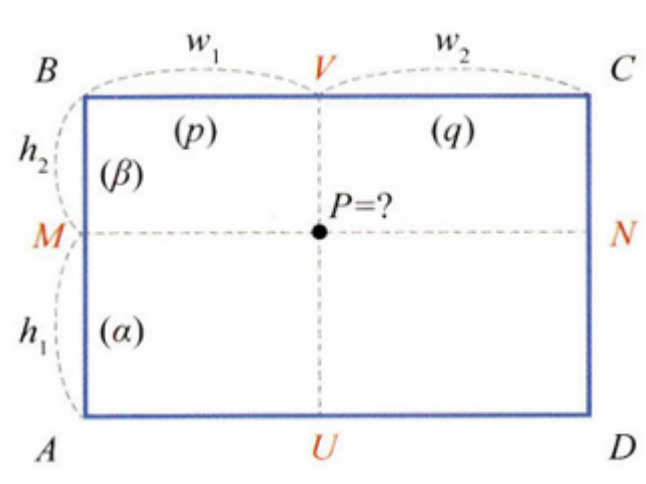
양선형 보간법

보간법: 화소 값을 할당받지 못한 영상(e.g. 영상의 빈 공간)의 품질은 안 좋을 수밖에 없는데, 이때 빈 화소에 값을 할당하여 좋은 품질의 영상을 만드는 방법

보간법에서는 선형 보간법과 양선형 보간법이 있다.

선형 보간법은 원시 영상의 화소 값 두 개를 사용해 원하는 좌표에서 새로운 화소 값을 계산하는 방법이다. 반면 양선형 보간법은 화소당 선형 보간을 세 번 수행하며, 새롭게 생성된 화소는 가장 가까운 화소 네 개에 가중치를 곱한 값을 합해서 얻는다.

다음 그림과 같이 직사각형의 네 꼭짓점에 값이 주어져 있을 때, 이 사각형 내부에 있는 임의의 점(P)에 대한 값을 추정해 본다.

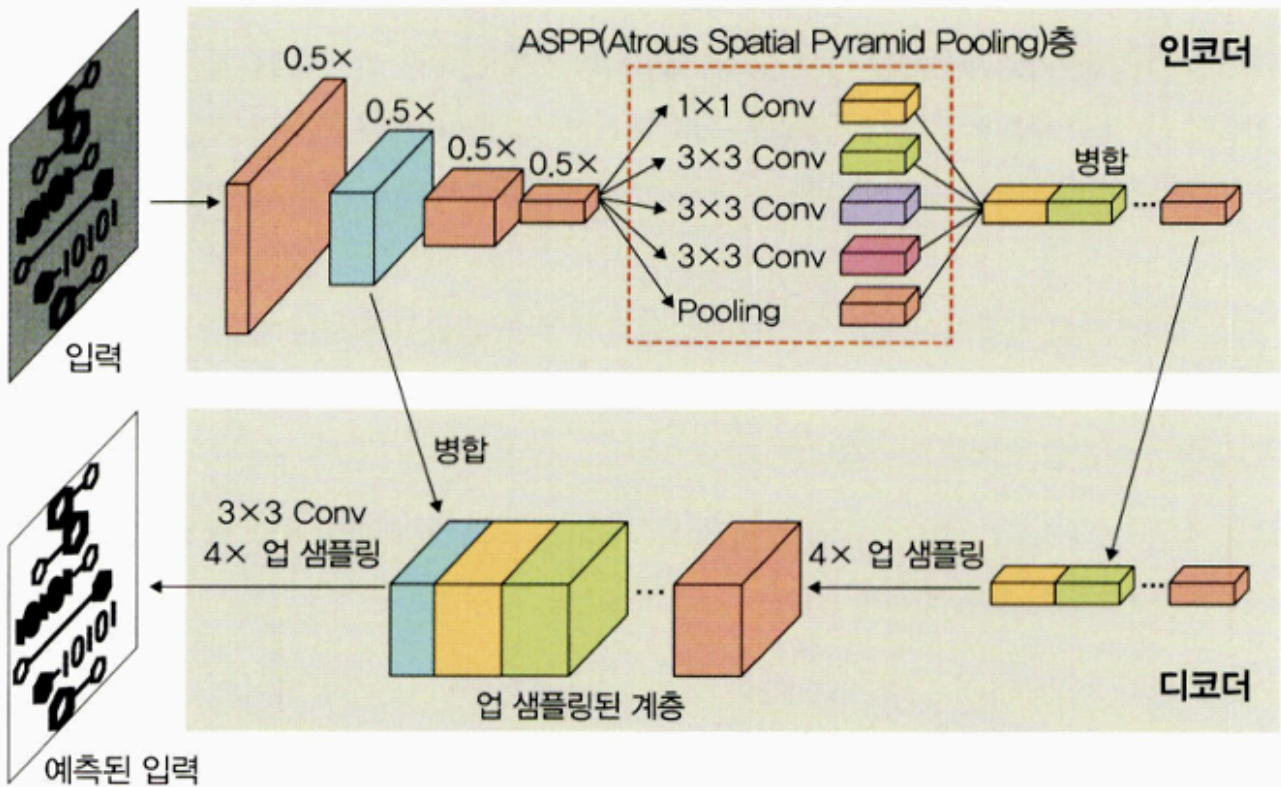


$$\begin{aligned}
 P &= q(\beta A + \alpha B) + p(\beta D + \alpha C) \\
 &= q\beta A + q\alpha B + p\beta D + p\alpha C
 \end{aligned}$$

6.3.5 DeepLabv3/DeepLabv3+

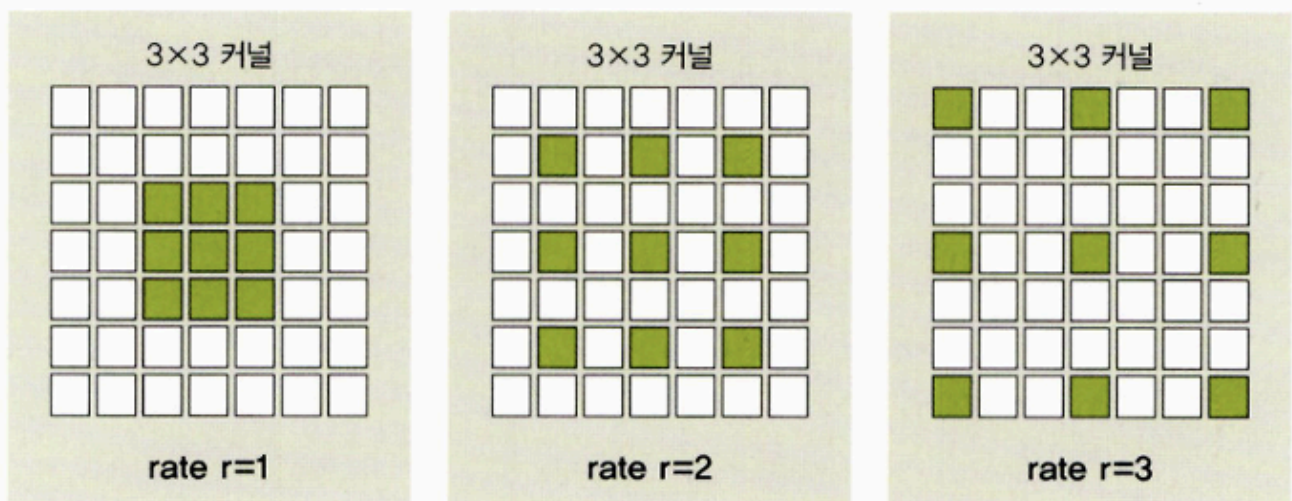
완전 연결층의 단점을 보완하기 위해 Atrous 합성곱을 사용하는 네트워크이다. 인코더와 디코더 구조를 가지며, 일반적으로 인코더-디코더 구조에서는 불가능했던 인코더에서 추출된 특성 맵의 해상도를 Atrous 합성곱을 도입하여 제어할 수 있도록 했다.

▼ 그림 6-54 DeepLab의 인코더-디코더 구조



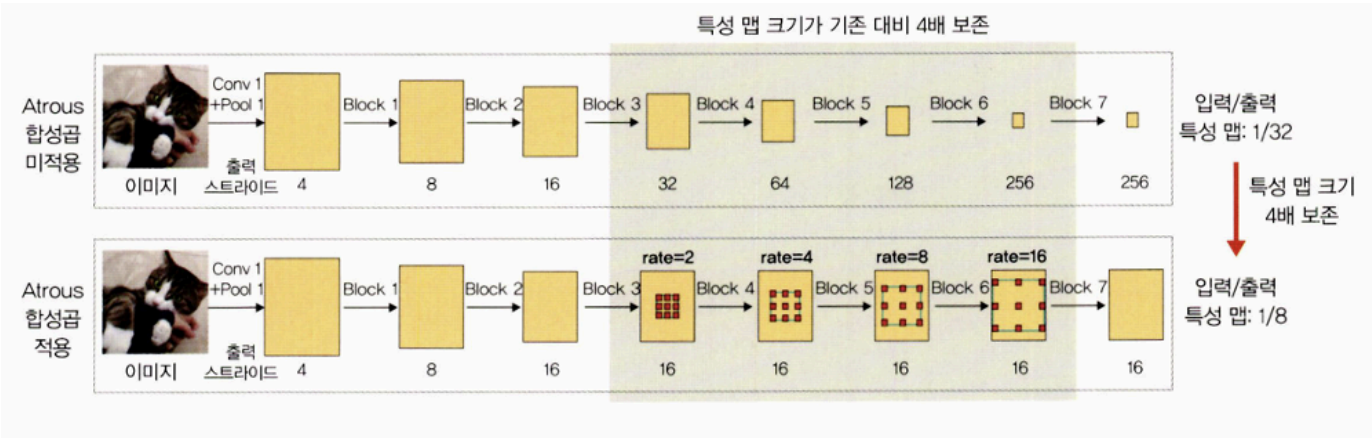
Atrous 합성곱은 다음 그림과 같이 필터 내부에 빈 공간을 둔 채로 작동한다. 얼마나 많은 빈공간을 가질지 결정하는 파라미터로 rate가 있다. rate $r=1$ 일 경우 기존 합성곱과 동일하게 빈 공간을 가지며, r 이 커질수록 빈 공간은 더 많아진다.

▼ 그림 6-55 Atrous 합성곱



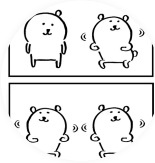
보통 이미지 분할에서 높은 성능을 내려면 수용 영역의 크기가 중요한데, 수용 영역을 확대하여 특성을 찾는 범위를 넓게 해 주기 때문이다. Atrous 합성곱을 활용하면 파라미터 수를 늘리지 않으면서도 수용 영역을 크게 키울 수 있기 때문에 이미지 분할 분야에서 많이 사용한다.

즉, 다음 그림과 같이 일반적인 CNN을 적용하면 출력은 입력에 비해 1/32로 줄어들지만, Atrous 합성곱을 적용하면 1/8로 줄어든다. 따라서 특성 맵 크기가 기존 대비 4배 보존된 것을 확인할 수 있다.



수용 영역

수용 영역: 외부 자극이 전체에 영향을 주는 것이 아니라 특정 영역에만 영향을 준다는 의미이다. 마찬가지로 영상에서 특정 위치에 있는 픽셀들은 그 주변에 있는 일부 픽셀과 연관성은 높지만 거리가 멀어질수록 그 영향은 감소하게 된다. 영상 전체 영역에 대해 서로 동일한 중요도를 부여하여 처리하는 대신, 특정 범위를 한정해서 처리하여 효과적으로 훈련을 수행하는 것이 수용 영역이다.



카사바

안녕하세요 :)



이전 포스트

0개의 댓글

댓글을 작성하세요

댓글 작성



Powered by
Stellate